PES UNIVERSITY • RR CAMPUS

FITZONE

Gym Membership Management System

PROJECT REPORT

Database Management Systems

SUBMITTED BY

Student	SRN	Programme
Ramalakshmi M N	PES1UG24AM219	CSE (AI & ML)
Priyanka B M	PES1UG24AM207	CSE (AI & ML)

PES University • RR Campus • Academic Year 2026–2027

Report prepared: 09 November 2026

Acknowledgement

The project team thanks PES University, RR Campus, and the Department of Computer Science and Engineering (AI & ML) for the opportunity to undertake the FITZONE Gym Membership Management System as a Database Management Systems project.

The team also acknowledges the open-source communities behind Next.js, React, TypeScript, MySQL, MongoDB and the supporting libraries used in this work, whose documentation and tools made the implementation and verification possible.

Abstract

FITZONE is a gym membership management system that centralises member records, membership plans, memberships, trainers, trainer assignments, attendance, payments and reports. It is built with Next.js, React and TypeScript, with server-side Route Handlers that use MySQL 8.0 as the relational system of record and MongoDB for selected activity-log documents.

The relational design links seven core tables with primary and foreign keys and protects data with unique, ENUM and CHECK constraints for attendance times, payment amounts and payment statuses. Reporting views and the stored procedure GetMemberPayments support administrative reporting. Authentication uses an auth_users table, bcrypt password verification and signed session tokens. MongoDB records attendance check-in events in an activity_logs collection.

The report documents the problem context, requirements, architecture, technology stack, database design, modules, API layer, workflows, testing, collaboration and results. Verification evidence includes a successful local administrator login, manual testing of the attendance, trainer, assignment, payment and report modules, a successful Next.js production build and successful GitHub Actions build checks. Items still outstanding, such as member-account authentication alignment, broader activity logging and the final pull-request review, are stated explicitly.

Keywords: gym management, MySQL, MongoDB, Next.js, TypeScript, referential integrity, activity logging.

Table of Contents

1. Introduction 1
1.1 Introduction to the System 1
1.2 Background and Motivation 1
1.3 Problem Statement 1
1.4 Project Objectives 1
1.5 Scope of the Project 2
1.6 Intended Users 2
1.7 Key Benefits of the System 2
1.8 Organisation of the Report 3
2. Existing System and Proposed Solution 4
2.1 Limitations of Traditional Gym Management 4
2.2 Problems with Manual Member and Payment Tracking 4
2.3 Need for Centralised Management 4
2.4 Proposed FITZONE Solution 4
2.5 Advantages of the Proposed System 5
3. System Requirements 6
3.1 Functional Requirements 6
3.2 Non-Functional Requirements 7
3.3 Hardware Requirements 7
3.4 Software and Technology Requirements 7
3.5 User Roles and Access Levels 8
3.6 System Constraints and Assumptions 8
4. System Architecture and Design 9
4.1 Overall System Architecture 9
4.2 Frontend Architecture 9
4.3 Backend and API Architecture 9
4.4 Database Architecture 10
4.5 Authentication Flow 10
4.6 MongoDB Integration 11
4.7 Interaction between Frontend, APIs, MySQL and MongoDB 11
4.8 Application Workflow 11
4.9 Configuration and Secrets 12
5. Technology Stack 13
6. Database Design and Implementation 14
6.1 Overview 14
6.2 Entity-Relationship Diagram 15
6.3 Table Definitions 16
6.4 Referential Integrity 19
6.5 Constraints and Validation 19
6.6 Views and Stored Procedures 19
6.7 Normalisation 20
6.8 MongoDB Activity Logging 20
7. Application Modules and Functionalities 21
7.1 Dashboard 21
7.2 Member Management 22
7.3 Membership Plan Management 23
7.4 Membership Management 23
7.5 Attendance Management 24
7.6 Trainer Management 25
7.7 Trainer Assignment Management 26
7.8 Payment Management 27
7.9 Reports and Analytics 28

7.10 Authentication and User Management 29
7.11 Activity Logging 29
8. API and Backend Implementation 30
8.1 Overview 30
8.2 Request and Response Handling 30
8.3 Database Operations 30
8.4 Validation and Error Handling 30
8.5 Authentication and Authorisation 31
8.6 Frontend–Backend Integration 31
9. User Interface and Application Workflow 32
9.1 Login and Registration 32
9.2 Dashboard Navigation 32
9.3 Member Management Workflow 32
9.4 Membership Workflow 33
9.5 Attendance Check-in and Check-out 33
9.6 Trainer Assignment Workflow 33
9.7 Payment Workflow 33
9.8 Reports Generation 33
9.9 Member Self-Service View 33
10. Testing and Validation 35
10.1 Testing Approach and Evidence Policy 35
10.2 Functional Testing 35
10.3 API Testing 36
10.4 Database Testing 36
10.5 Integration Testing 37
10.6 Manual Testing 37
10.7 Build Verification and GitHub Actions Checks 38
10.8 Validation and Error-Handling Tests 38
10.9 Summary of Test Status 38
11. Version Control and Team Collaboration 40
11.1 Git and GitHub Workflow 40
11.2 Repository Structure 40
11.3 Branching Strategy 40
11.4 Pull Request Workflow 40
11.5 Code Review 40
11.6 Continuous Integration 41
11.7 Team Collaboration and Integration Process 41
11.8 Member 2 Contribution within the Overall Project 41
11.9 Integration Considerations 41
12. Results and Discussion 42
12.1 Overall System Functionality 42
12.2 Completed Modules 42
12.3 Integration Results 42
12.4 Testing Outcomes 42
12.5 Benefits Achieved 43
12.6 Current Limitations 43
13. Future Enhancements 44
14. Conclusion 45
References 46
Appendix A: Database Scripts and Queries 47
A.1 MySQL Statements 47
A.2 MongoDB Shell Commands 47
A.3 Authentication Table Specification 47
Appendix B: Supporting Evidence 48

B.1 Database Inspection Screenshots....48
B.2 Diagram Files....48
B.3 Test Evidence Summary....49
Appendix C: API Route Data-Access Summary....50
Appendix D: Pre-Submission Checklist....51

List of Figures

Figure 4.1: High-level architecture of FITZONE....9
Figure 4.2: Authentication flow....10
Figure 4.3: Level-1 data flow diagram....11
Figure 4.4: Application workflow....12
Figure 6.1: Entity-relationship diagram of the Gym Membership Management System (team ERD)....15
Figure 7.1: Administrator dashboard....21
Figure 7.2: Members Management page....22
Figure 7.3: Membership Management page....24
Figure 7.4: Attendance Management page....25
Figure 7.5: Trainers Management page, including the assignment section....26
Figure 7.6: Payments Management page....27
Figure 7.7: Reports and Analytics page....28
Figure 9.1: Dashboard sidebar and Quick Navigation cards....32
Figure 9.2: Member Dashboard....34
Figure B.1: SHOW TABLES output for gym_membership_db....48
Figure B.2: DESCRIBE output for Member, Membership_Plan, Membership and Trainer....48

List of Tables

Table 1.1: Intended users of FITZONE..... 2
Table 2.1: Traditional approach compared with FITZONE..... 5
Table 3.1: Functional requirements..... 6
Table 3.2: Non-functional requirements and how they are addressed..... 7
Table 3.3: Software requirements..... 7
Table 3.4: User roles and observed access..... 8
Table 5.1: Technologies used in FITZONE..... 13
Table 6.1: Database object summary..... 14
Table 6.2: Relationships..... 16
Table 6.3: Member table (from DESCRIBE)..... 16
Table 6.4: Membership_Plan table (from DESCRIBE)..... 16
Table 6.5: Membership table (from DESCRIBE)..... 17
Table 6.6: Attendance table (from the team ER diagram and report text)..... 17
Table 6.7: Trainer table (from DESCRIBE)..... 17
Table 6.8: Trainer_Assignment table (from the team ER diagram)..... 18
Table 6.9: Payment table (from the team ER diagram and report text)..... 18
Table 6.10: auth_users table (from the setup-script description)..... 18
Table 6.11: Constraints and validation rules..... 19
Table 6.12: Reporting objects..... 19
Table 6.13: MongoDB verification observed on the local service..... 20
Table 7.1: Dashboard module profile..... 21
Table 7.2: Member management module profile..... 22
Table 7.3: Membership plan module profile..... 23
Table 7.4: Membership management module profile..... 23
Table 7.5: Attendance module profile..... 24
Table 7.6: Trainer management module profile..... 25
Table 7.7: Trainer assignment module profile..... 26
Table 7.8: Payment module profile..... 27
Table 7.9: Reports module profile..... 28
Table 7.10: Authentication module profile..... 29
Table 7.11: Activity logging module profile..... 29
Table 8.1: API route directories and documented responsibilities..... 30
Table 10.1: Status legend..... 35
Table 10.2: Functional test cases..... 35
Table 10.3: API test cases..... 36
Table 10.4: Database test cases..... 36
Table 10.5: Integration test cases..... 37
Table 10.6: MongoDB checks..... 37
Table 10.7: Build and CI test cases..... 38
Table 10.8: Validation test cases..... 38
Table 10.9: Test status summary..... 39
Table 11.1: Team members and responsibilities..... 41
Table 12.1: Module status and evidence..... 42
Table 13.1: Proposed enhancements..... 44
Table B.1: Observed evidence and interpretation..... 49
Table C.1: Route to data-store summary..... 50
Table D.1: Items to complete or confirm before submission..... 51

1. Introduction

1.1 Introduction to the System

FITZONE is a web-based Gym Membership Management System that brings the everyday records of a gym into one application. Member profiles, membership plans, memberships, trainers, trainer assignments, attendance, payments and administrative reports are managed through a single interface and stored in a MySQL relational database named `gym_membership_db`. A MongoDB collection, `activity_logs`, stores selected operational events as documents.

The application is built with Next.js, React and TypeScript. Server-side logic is implemented as Next.js Route Handlers that read and write MySQL through the `mysql2` driver and write activity events to MongoDB through the official `mongodb` driver. FITZONE was developed by a two-member team in the Department of Computer Science and Engineering (AI & ML) at PES University, RR Campus, as a Database Management Systems project.

1.2 Background and Motivation

A gym maintains a long-running relationship with every member. Each member has a chosen plan, a membership period, a payment history, a record of visits and, often, an assigned trainer. When these facts are kept in paper registers or separate spreadsheets, they drift out of step with each other, and simple questions such as who is due for renewal this week or which payments are still pending become slow to answer.

A relational database is a natural fit for this domain because the entities (members, plans, memberships, payments, trainers, attendance) have well-defined relationships and integrity rules. Operational events such as a check-in, on the other hand, are append-only records whose fields may vary. FITZONE therefore combines MySQL for the structured core with MongoDB for flexible, document-style activity logging.

1.3 Problem Statement

Manual or disconnected record keeping can cause duplicate records, difficulty tracking membership periods, inconsistent payment records and slow retrieval of attendance or trainer-assignment information. FITZONE addresses this by providing a shared application interface backed by structured database tables, integrity constraints and reporting queries.

1.4 Project Objectives

- Maintain member and membership information in MySQL.
- Record attendance with check-in and check-out validation.
- Maintain trainer details and trainer-to-member assignments.
- Record payments with valid amounts and controlled statuses.
- Provide dashboards and reports for administrative use.
- Protect access through authenticated sign-in using hashed passwords and signed session tokens.
- Store selected operational activity events in MongoDB.

- Use Git, GitHub and GitHub Actions to integrate the work of both team members and verify that the application builds.

1.5 Scope of the Project

The scope below reflects what is evidenced in the project materials (application screenshots, database inspection output, build and CI results).

1.5.1 In scope

- Dashboard, Member Management, Membership Management, Trainer Management, Trainer Assignment, Attendance Management, Payment Management and Reports.
- Authenticated sign-in with an administrator account and a Member Dashboard showing a member's own memberships, attendance and payments.
- MySQL schema with primary keys, foreign keys, unique and check constraints, reporting views and the stored procedure `GetMemberPayments`.
- MongoDB activity logging of attendance check-in events.

1.5.2 Outside the verified scope

- No vector database, embedding pipeline or similarity search is part of FITZONE. MongoDB is used only as a document store for activity logs.
- MongoDB logging is verified for attendance only; logging for trainer, assignment and payment actions has not been established.
- Member-account authentication is not yet aligned with the shared Member data (see Chapter 12).

1.6 Intended Users

Table 1.1: Intended users of FITZONE

User	Role in the system	Typical interactions
Gym administrator / front-desk staff	Primary operator. Signs in with an administrator account.	Registers members, assigns memberships, records attendance and payments, manages trainers and assignments, reads dashboard and reports.
Member	Secondary user with a personal, read-only view (Member Dashboard).	Views own member ID, plan, membership status, memberships, attendance and payment history.
Trainer	Maintained as a record (directory entry and assignments). A trainer sign-in is not part of the verified implementation.	Appears in the Trainer Directory and in assignments with members.

1.7 Key Benefits of the System

- **Single source of truth:** one relational database replaces scattered registers and sheets.
- **Integrity by design:** primary keys, foreign keys, unique keys and CHECK constraints reject invalid or duplicate data at the database layer.

- **Faster retrieval:** searchable directories, summary cards, reporting views and a stored procedure for member payments.
- **Operational traceability:** attendance check-ins are mirrored as activity documents in MongoDB.
- **Safer credential handling:** passwords are verified against bcrypt hashes, and secrets are kept in a local environment file that is not committed.

1.8 Organisation of the Report

Chapters 2 and 3 describe the problem context and requirements. Chapters 4 to 6 explain the architecture, technology stack and database. Chapters 7 to 9 describe the application modules, API layer and user workflows. Chapters 10 to 12 cover testing, collaboration and results. Chapters 13 and 14 present future work and the conclusion, followed by references and appendices.

2. Existing System and Proposed Solution

2.1 Limitations of Traditional Gym Management

- Member details are held in paper registers or individual spreadsheets, so the same person can be entered more than once with different spellings or contact details.
- Nothing enforces relationships between records. A payment can be recorded against a membership that does not exist, or a trainer can be linked to a person who is not a member.
- Information is hard to search. Finding a member's current plan, last visit and payment status may require checking several files.
- Reports are compiled by hand, which is slow and error-prone.

2.2 Problems with Manual Member and Payment Tracking

- **Membership periods:** end dates are calculated by hand from the plan duration, and expiries are noticed late.
- **Payments:** amounts, dates, methods and statuses (pending, completed, failed, refunded) are recorded inconsistently, which makes revenue totals unreliable.
- **Attendance:** sign-in sheets cannot be validated, so a check-out earlier than a check-in goes unnoticed.
- **Trainer assignments:** it is unclear which members each trainer is responsible for, and for how long.

2.3 Need for Centralised Management

A centralised system lets every part of the gym operation refer to the same member and membership records. When attendance, payments and trainer assignments all point to the same Member and Membership keys, summary figures on the dashboard and in reports are produced from one consistent data set rather than reconciled manually.

2.4 Proposed FITZONE Solution

FITZONE provides a web interface for administrators and a personal dashboard for members, supported by a MySQL database with enforced relationships and a MongoDB collection for activity logging. Table 2.1 contrasts the traditional approach with FITZONE.

Table 2.1: Traditional approach compared with FITZONE

Aspect	Traditional approach	FITZONE
Member records	Scattered registers and sheets; duplicates likely.	Single `Member` table; email and phone are unique.
Membership tracking	End dates calculated manually.	Plan duration gives a calculated end date; status is Active, Expired or Cancelled; the dashboard shows memberships expiring within 7 days.
Attendance	Paper sign-in; no validation.	Check-in and optional check-out records; a CHECK constraint and API validation protect the times.
Payments	Informal notes.	Payment rows linked to memberships; non-negative amounts; controlled status values.
Trainers	Informal arrangements.	Trainer directory and `Trainer_Assignment` table; assigned-member counts.
Reporting	Manual compilation.	Dashboard cards, Reports page, SQL reporting views and `GetMemberPayments`.
Activity history	None.	MongoDB `activity_logs` documents (verified for attendance).

2.5 Advantages of the Proposed System

- One interface for all gym-operation records, reducing duplicate entry.
- Constraint-backed data quality: invalid payment amounts, invalid statuses and inconsistent attendance times are rejected by the database.
- Immediate visibility of totals and statuses through live database data.
- Clear separation between structured data (MySQL) and event history (MongoDB).
- A repeatable build verified locally and by GitHub Actions.

3. System Requirements

3.1 Functional Requirements

Table 3.1 lists the functional requirements that the implemented system satisfies, with the evidence level of each. “UI observed” means the behaviour is visible in the supplied application screenshots; “Manual” means it was tested manually by the team; “Verified” means a specific check was recorded.

Table 3.1: Functional requirements

ID	Requirement	Module	Evidence
FR-01	An administrator can sign in with email and password; the password is checked against a stored hash and a signed session is created.	Authentication	Verified (local admin login)
FR-02	The dashboard shows total members, active memberships, memberships expiring within 7 days, total trainers, completed and pending payments, a searchable member directory and quick navigation.	Dashboard	UI observed
FR-03	An administrator can add a member (name, email, phone, gender) and search the member directory by name or email.	Members	UI observed
FR-04	An administrator can assign a membership by choosing a member, a plan and a start date; the end date is calculated; membership status can be viewed and changed.	Memberships	UI observed
FR-05	An administrator can record attendance (member, date, check-in, optional check-out) and search attendance records.	Attendance	Manual
FR-06	An administrator can add trainers (name, specialization, phone, email) and view the Trainer Directory.	Trainers	Manual
FR-07	An administrator can assign a member to a trainer and view current assignments.	Assignments	Manual
FR-08	An administrator can record a payment against a membership (amount, date, method, status) and view payment history and revenue cards.	Payments	Manual
FR-09	The Reports page shows summary cards, a daily attendance report and a payment report with a status filter.	Reports	Manual
FR-10	A member can view a personal dashboard of memberships, attendance and payment history.	Member Dashboard	UI observed; member authentication alignment pending
FR-11	A successful attendance record produces a Check-in activity document in MongoDB.	Activity logging	Verified for attendance

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements and how they are addressed

ID	Category	Requirement	How addressed
NFR-01	Data integrity	Invalid or duplicate data must be rejected.	Primary, foreign, unique and CHECK constraints in MySQL.
NFR-02	Security	Credentials and secrets must be protected.	bcrypt password hashes (`bcryptjs`), signed tokens (`jose`), `AUTH_SECRET` and database credentials kept in `.env.local`, which is not committed.
NFR-03	Reliability	A logging fault must not be confused with a database fault.	MongoDB logging errors are handled separately from the MySQL write.
NFR-04	Usability	Common tasks should be quick to perform.	Form-based entry, summary cards, status badges, search boxes and a persistent sidebar.
NFR-05	Maintainability	Code should be organised and verifiable.	TypeScript, Route Handlers per module, shared `lib/db.ts` and `lib/mongodb.ts`, and a CI build.
NFR-06	Portability	The system should run on a developer machine.	Node.js with local MySQL and MongoDB services; configuration through environment variables.
NFR-07	Performance	Not specified.	No benchmark results were recorded, so no performance claim is made.

3.3 Hardware Requirements

Hardware requirements are not documented in the project materials. FITZONE was developed and verified on local development machines running Node.js, a local MySQL 8.0 server and a local MongoDB service. Any computer able to run these three services and a modern web browser is expected to be sufficient for demonstration-scale use; this is an expectation rather than a measured result.

3.4 Software and Technology Requirements

Table 3.3: Software requirements

Component	Requirement
Runtime	Node.js with npm (dependencies in `package.json` and `package-lock.json`).
Application framework	Next.js 16.4.2 with React and TypeScript.
Relational database	MySQL 8.0; database `gym_membership_db`.
Document database	MongoDB; database `gym_membership_db`, collection `activity_logs`.
Libraries	`mysql2`, `mongodb`, `bcryptjs`, `jose`.
Configuration	Local `.env.local` holding database credentials and `AUTH_SECRET` (never committed).
Client	A modern web browser.
Tooling	VS Code, PowerShell, MySQL CLI, mongosh, Git and GitHub.

3.5 User Roles and Access Levels

Table 3.4: User roles and observed access

Role	Observed access	Notes
Administrator	Dashboard, Members, Memberships, Trainers (with assignments), Attendance, Payments and Reports.	A local administrator account in `auth_users` was verified by a successful login.
Member	Personal Member Dashboard (own memberships, attendance, payments).	`auth_users.role` and `member_id` link an account to a member. Alignment with shared Member data is pending.

To confirm. The supplied evidence does not include route-level authorisation rules. Whether every administrative page and API route rejects unauthenticated or member-role requests should be confirmed from the source code before this report is submitted.

3.6 System Constraints and Assumptions

- The database and application are configured locally; shared deployment settings are not documented.
- The authentication table was set up by a local script (`auth_users_setup.sql`). The other team member used a separate local database, so the two authentication setups must be compared before integration.
- `CREATE TABLE IF NOT EXISTS` does not add constraints to a table that already exists. The integration script must therefore be reviewed against the target schema before it is run on a shared database.
- Table names use mixed case (for example `Member`, `Membership_Plan` and `attendance`, `payment`). On MySQL servers where table names are case-sensitive, statements must use the exact case.
- MongoDB logging is assumed to be best-effort supplementary data; MySQL is the system of record.

4. System Architecture and Design

4.1 Overall System Architecture

FITZONE follows a three-tier structure. The client tier is a web browser. The application tier is a single Next.js application on Node.js that contains both the React pages and the server-side Route Handlers. The data tier consists of MySQL, which holds the structured records, and MongoDB, which holds activity-log documents. Figure 4.1 shows the components and their connections.

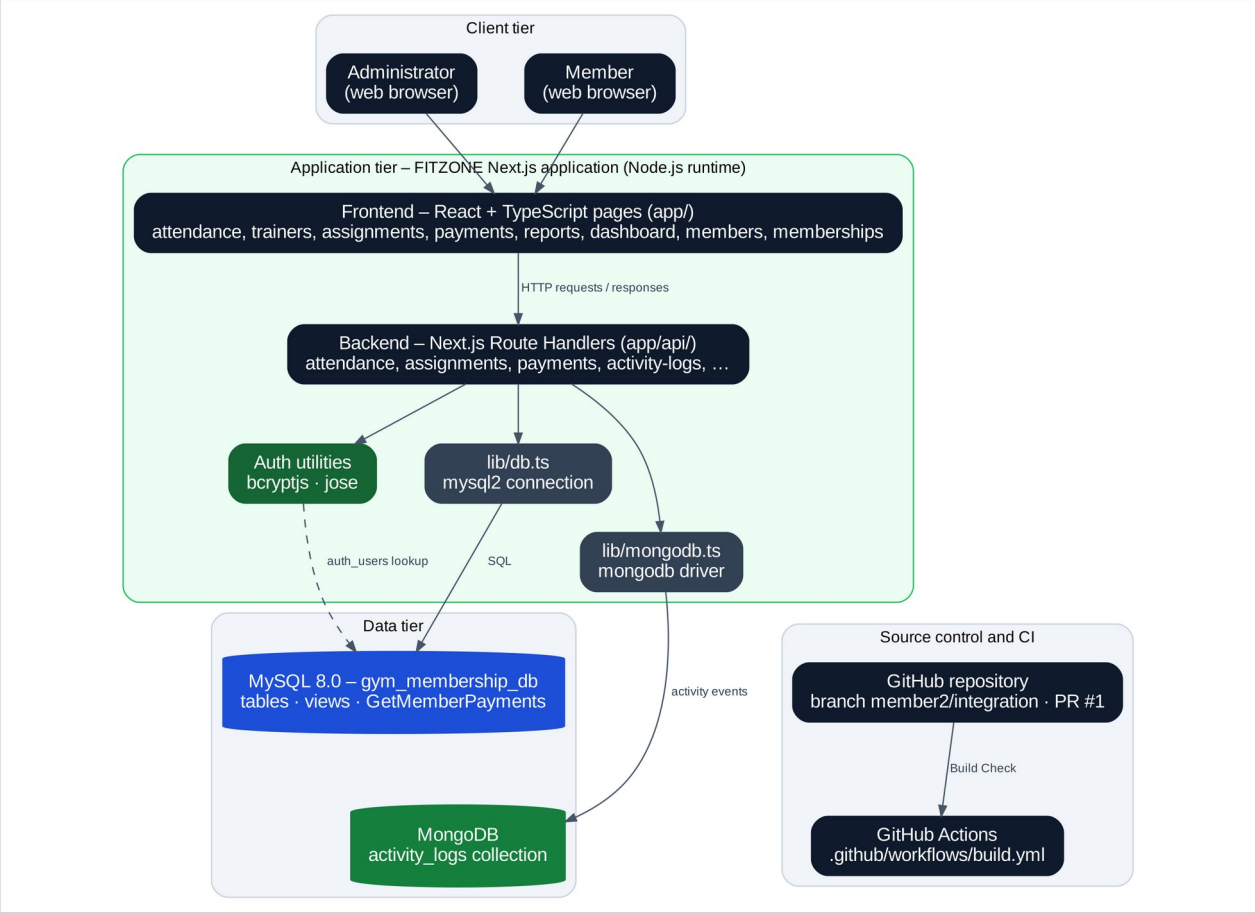


Figure 4.1: *High-level architecture of FITZONE*

4.2 Frontend Architecture

The frontend is built with Next.js, React and TypeScript. Each module has its own page under the `app/` directory, including `attendance/`, `trainers/`, `assignments/`, `payments/` and `reports/`, together with the dashboard, member and membership pages. Pages present forms, tables, summary cards and search boxes, and communicate with the backend through the application's API routes. The interface uses a dark navy sidebar with a green active-item highlight, light content cards and coloured status badges.

4.3 Backend and API Architecture

Server-side logic is implemented as Next.js Route Handlers under `app/api/`. The documented route directories are `attendance/`, `assignments/`, `payments/` and `activity-logs/`, with further routes for

the other modules. Handlers obtain a MySQL connection from `lib/db.ts` (built on `mysql2`) and a MongoDB connection from `lib/mongodb.ts` (built on the `mongodb` driver). Chapter 8 describes the routes in more detail.

4.4 Database Architecture

MySQL 8.0 hosts the database `gym_membership_db`. It contains seven core tables (`Member`, `Membership_Plan`, `Membership`, `Trainer`, `Trainer_Assignment`, `Attendance`, `Payment`), the authentication table `auth_users`, reporting views and the stored procedure `GetMemberPayments`. MongoDB hosts the collection `activity_logs` in a database of the same name. Chapter 6 gives the full design.

4.5 Authentication Flow

Authentication relies on the `auth_users` table, `bcryptjs` for verifying passwords against stored hashes and `jose` for creating a signed session token, with the signing secret read from `AUTH_SECRET`. Figure 4.2 shows the flow assembled from these documented components.



Figure 4.2: *Authentication flow*

> **Note.** The diagram shows the steps implied by the documented components. The exact route name, where the session token is stored and how each page enforces the role are implementation details that were not part of the supplied evidence.

4.6 MongoDB Integration

When an attendance record is created, the attendance API inserts the row in MySQL and then attempts to write a corresponding Check-in document to the `activity_logs` collection. MongoDB errors are handled separately, so the relational record remains the source of truth. A stored document contains `member_id`, `activity`, `timestamp` and `details` (which includes the attendance ID).

4.7 Interaction between Frontend, APIs, MySQL and MongoDB

Figure 4.3 is a Level-1 data flow diagram of the system. Administrators and members enter through authentication; administrators exercise the management processes, which read and write MySQL; attendance additionally emits an event to MongoDB; reports and the member view read from the stored data.

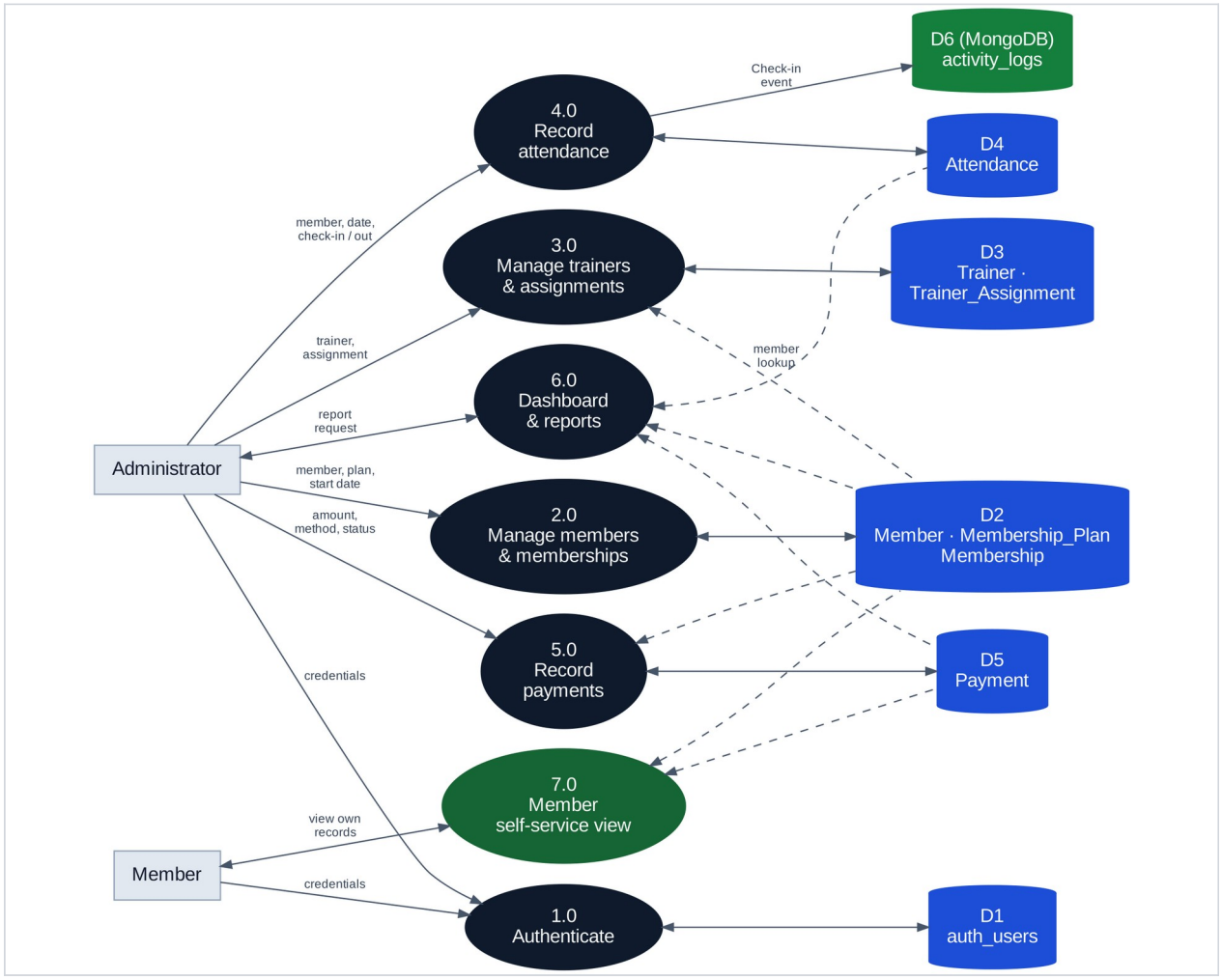


Figure 4.3: *Level-1 data flow diagram*

4.8 Application Workflow

Figure 4.4 summarises the common path through the application: sign in, open the dashboard, choose a module, submit a form, have the request validated, persist the change in MySQL (with the extra MongoDB step for attendance) and see the refreshed list or totals. Reports are read-only.

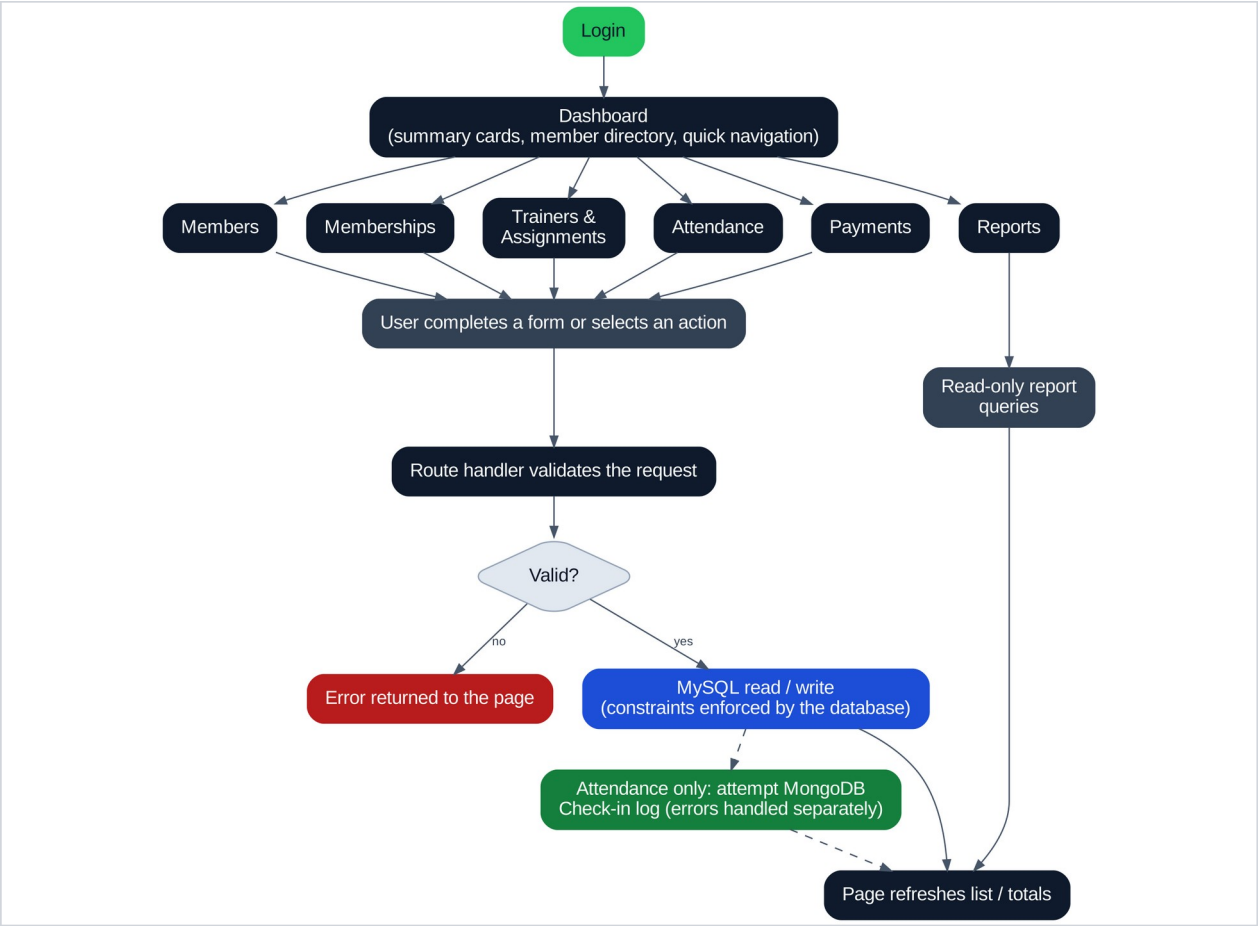


Figure 4.4: *Application workflow*

4.9 Configuration and Secrets

Database credentials and `AUTH_SECRET` are stored in a local `.env.local` file. This file must not be committed to the repository or shown in screenshots.

5. Technology Stack

Table 5.1 lists the technologies documented for FITZONE and the role each plays. Versions are given only where they are recorded in the project materials.

Table 5.1: Technologies used in FITZONE

Technology	Version	Role in the system	Where used
Next.js	16.4.2	Full-stack framework: pages, Route Handlers and the production build (30 static pages generated in the verified build).	`app/`, `app/api/`
React	Not recorded	Component-based user interface: forms, tables, cards.	`app/` pages
TypeScript	Not recorded	Typed source code; type checks run during the build.	Whole project
Node.js	Not recorded	Server runtime for Next.js and the database drivers.	Runtime
MySQL	8.0	Primary relational database `gym_membership_db`: tables, constraints, views, stored procedure.	SQL scripts, `lib/db.ts`
mysql2	Not recorded	MySQL driver used by the server code.	`lib/db.ts`
MongoDB	Not recorded	Document database for the `activity_logs` collection.	`lib/mongodb.ts`
mongodb (driver)	Not recorded	Official Node.js driver for MongoDB.	`lib/mongodb.ts`
bcryptjs	Not recorded	Verification of password hashes.	Authentication
jose	Not recorded	Creation and verification of signed session tokens.	Authentication
npm	Not recorded	Dependency management.	`package.json`, `package-lock.json`
ESLint	Not recorded	Static code analysis. A repository-wide run still reports issues in other or shared files.	Development
Git and GitHub	—	Version control; branch `member2/integration`; Pull Request #1.	Repository
GitHub Actions	—	Automated “Build Check” workflow.	`.github/workflows/build.yml`
VS Code, PowerShell, MySQL CLI, mongosh	—	Development, database inspection and verification.	Developer workstations

Not part of the verified stack. No vector database, embedding pipeline or similarity query is used. Tailwind CSS is not confirmed by the supplied evidence; it is deliberately not listed and should be added only if `package.json` shows it as a dependency.

6. Database Design and Implementation

6.1 Overview

The relational design is centred on the `Member` table. Memberships link members to plans, payments link to memberships, attendance and trainer assignments link to members, and trainers link to assignments. The authentication table optionally links an account to a member. Column definitions below are taken from the `DESCRIBE` output where it was captured (Member, Membership_Plan, Membership, Trainer) and from the team's ER diagram and report text for the remaining tables.

Table 6.1: Database object summary

Object	Kind	Purpose	Primary key	Foreign keys
Member	Table	Member profiles	member_id	—
Membership_Plan	Table	Plan catalogue (duration, price)	plan_id	—
Membership	Table	A member's plan and period	membership_id	member_id → Member plan_id → Membership_Plan
Attendance	Table	Check-in and check-out records	attendance_id	member_id → Member
Trainer	Table	Trainer directory	trainer_id	—
Trainer_Assignment	Table	Trainer-to-member assignments	assignment_id	trainer_id → Trainer member_id → Member
Payment	Table	Payments against memberships	payment_id	membership_id → Membership
auth_users	Table (setup script)	Login accounts	user_id	member_id → Member (ON DELETE SET NULL)
daily_attendance_report	View	Attendance reporting	—	—
payment_report	View	Payment reporting	—	—
member_membership_report	Reporting object	Listed in the schema output; definition not captured	—	—
GetMemberPayments	Stored procedure	Returns a member's payment records	—	—

Evidence note. The supplied `SHOW TABLES` output lists nine objects: attendance, auth_users, Member, member_membership_report, Membership, Membership_Plan, payment, trainer and trainer_assignment. `daily_attendance_report` and `payment_report` are queried in the project report but do not appear in that particular listing, so their presence should be re-checked with `SHOW FULL TABLES`.

6.2 Entity-Relationship Diagram

Figure 6.1 shows the entity-relationship diagram supplied by the project team. Rectangles are entities (tables), diamonds are relationships, ovals are attributes, and cardinalities are written as 1 and N. Primary keys are underlined and foreign keys are in italics. The MongoDB `ACTIVITY_LOG` collection is drawn inside a dashed box, linked to `AUTH_USERS` by the dashed relationship “generates”.

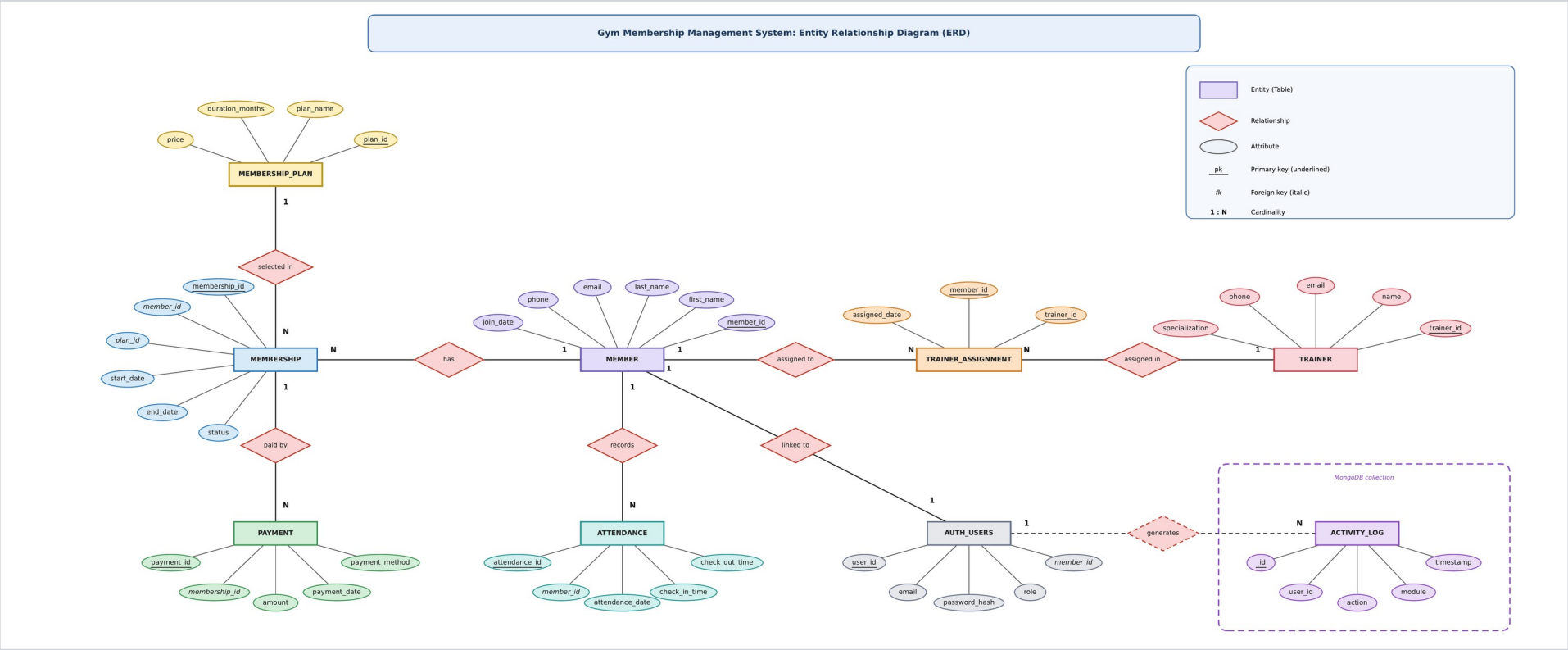


Figure 6.1: *Entity-relationship diagram of the Gym Membership Management System (team ERD)*

Table 6.2: Relationships

Parent (one)	Child (zero or many)	Foreign key	Meaning
Member	Membership	Membership.member_id	A member can hold many memberships over time.
Membership_Plan	Membership	Membership.plan_id	A plan can be chosen for many memberships.
Membership	Payment	Payment.membership_id	A membership can have many payments.
Member	Attendance	Attendance.member_id	A member can have many attendance records.
Member	Trainer_Assignment	Trainer_Assignment.member_id	A member can be assigned through many assignment rows.
Trainer	Trainer_Assignment	Trainer_Assignment.trainer_id	A trainer can take many assignments.
Member	auth_users	auth_users.member_id (unique)	At most one login account per member; ON DELETE SET NULL.

6.3 Table Definitions

6.3.1 Members

Table 6.3: Member table (from DESCRIBE)

Column	Data type	Key	Constraints and notes
member_id	INT	PK	NOT NULL, AUTO_INCREMENT
full_name	VARCHAR(100)		NOT NULL
email	VARCHAR(100)	UNI	NOT NULL, unique
phone	VARCHAR(15)	UNI	NOT NULL, unique
gender	VARCHAR(20)		NULL allowed
join_date	DATE		NOT NULL, default CURDATE()
created_at	TIMESTAMP		NULL allowed, default CURRENT_TIMESTAMP

6.3.2 Membership Plans

Table 6.4: Membership_Plan table (from DESCRIBE)

Column	Data type	Key	Constraints and notes
plan_id	INT	PK	NOT NULL, AUTO_INCREMENT
plan_name	VARCHAR(50)	UNI	NOT NULL, unique
duration_months	INT		NOT NULL
price	DECIMAL(10,2)		NOT NULL
description	VARCHAR(255)		NULL allowed

The application shows four plans: Monthly (1 month, ₹1,500), Quarterly (3 months, ₹4,500), Half-Yearly (6 months, ₹7,500) and Yearly (12 months, ₹12,000).

6.3.3 Memberships

Table 6.5: Membership table (from DESCRIBE)

Column	Data type	Key	Constraints and notes
membership_id	INT	PK	NOT NULL, AUTO_INCREMENT
member_id	INT	MUL	NOT NULL, foreign key to Member
plan_id	INT	MUL	NOT NULL, foreign key to Membership_Plan
start_date	DATE		NOT NULL
end_date	DATE		NOT NULL (calculated from the plan duration in the interface)
status	ENUM('Active','Expired','Cancelled')	MUL	NOT NULL, default 'Active', indexed

6.3.4 Attendance

Table 6.6: Attendance table (from the team ER diagram and report text)

Column	Data type	Key	Constraints and notes
attendance_id	INT	PK	Primary key
member_id	INT	FK	References Member(member_id)
attendance_date	DATE		Date of the visit
check_in_time	TIME		Check-in time
check_out_time	TIME		NULL allowed; CHECK (check_out_time IS NULL OR check_out_time >= check_in_time)

6.3.5 Trainers

Table 6.7: Trainer table (from DESCRIBE)

Column	Data type	Key	Constraints and notes
trainer_id	INT	PK	NOT NULL, AUTO_INCREMENT
trainer_name	VARCHAR(100)		NOT NULL
specialization	VARCHAR(100)		NULL allowed
phone	VARCHAR(15)	UNI	NULL allowed, unique
email	VARCHAR(100)	UNI	NULL allowed, unique

6.3.6 Trainer Assignments

Table 6.8: Trainer_Assignment table (from the team ER diagram)

Column	Data type	Key	Constraints and notes
assignment_id	INT	PK	Primary key
trainer_id	INT	FK	References Trainer(trainer_id)
member_id	INT	FK	References Member(member_id)
assigned_date	DATE		Date the assignment started
end_date	DATE		NULL allowed (open-ended assignment)

6.3.7 Payments

Table 6.9: Payment table (from the team ER diagram and report text)

Column	Data type	Key	Constraints and notes
payment_id	INT	PK	Primary key
membership_id	INT	FK	References Membership(membership_id)
amount	DECIMAL(10,2)		CHECK (amount >= 0)
payment_date	DATE		Date of payment
payment_method	VARCHAR(30)		For example UPI
payment_status	VARCHAR(20)		CHECK status IN ('Pending', 'Completed', 'Failed', 'Refunded')

To confirm. The report text writes the status check on a column called `status`, while the ER diagram names the column `payment_status`. Confirm the exact column name against `member2_integration.sql`. For Attendance, Trainer_Assignment and Payment, `DESCRIBE` output was not among the supplied evidence, so data types follow the team ER diagram.

6.3.8 Authentication Users

The local script `auth_users_setup.sql` creates `auth_users` with the columns below. Data types were not captured in the supplied evidence and are therefore not stated.

Table 6.10: auth_users table (from the setup-script description)

Column	Constraints and notes
user_id	Primary key
full_name	Display name
email	Unique key; used as the login identifier
password_hash	bcrypt hash of the password
role	Role of the account (administrator or member)
member_id	Unique key; foreign key to Member(member_id), ON DELETE SET NULL
created_at	Creation timestamp

6.4 Referential Integrity

Foreign keys ensure that a membership refers to an existing member and plan, a payment to an existing membership, attendance to an existing member, and an assignment to an existing trainer and member. The only delete rule captured in the evidence is on `auth_users.member_id`, which is set to NULL if the member is deleted, so a login account is never left pointing at a missing member. Delete and update rules for the other keys were not captured and should be read from the SQL scripts.

6.5 Constraints and Validation

Table 6.11: Constraints and validation rules

Constraint	Table	Rule	Layer
Primary key, auto-increment	All tables	Surrogate integer identifiers.	Database
UNIQUE	Member	email and phone cannot repeat.	Database
UNIQUE	Membership_Plan	plan_name cannot repeat.	Database
UNIQUE	Trainer	phone and email cannot repeat (NULL allowed).	Database
UNIQUE	auth_users	email and member_id cannot repeat.	Database
ENUM and default	Membership	status limited to Active, Expired, Cancelled; default Active.	Database
CHECK	Attendance	check-out must be NULL or not earlier than check-in.	Database and API
CHECK	Payment	amount must be non-negative.	Database
CHECK	Payment	status must be Pending, Completed, Failed or Refunded.	Database
Foreign keys	See Table 6.2	Child rows must reference an existing parent row.	Database

6.6 Views and Stored Procedures

Table 6.12: Reporting objects

Object	Kind	Documented use
daily_attendance_report	View	Queried with `SELECT * FROM daily_attendance_report;` for attendance reporting.
payment_report	View	Queried with `SELECT * FROM payment_report;` for payment reporting.
member_membership_report	Reporting object	Appears in the schema listing; its definition was not captured.
GetMemberPayments(member_id)	Stored procedure	Accepts a member identifier and returns that member's payment records: `CALL GetMemberPayments(1);`.

To confirm. View definitions are not reproduced because they were not part of the supplied evidence. Whether the Reports page reads these views directly or aggregates through route handlers should be confirmed from the source.

6.7 Normalisation

The following analysis is based on the captured schema.

- **First normal form:** every column holds a single atomic value, and each table has a primary key.
- **Second normal form:** every table uses a single-column surrogate key, so partial dependencies cannot occur.
- **Third normal form:** non-key attributes depend on the key and not on one another. Plan price and duration live only in `Membership_Plan`, member contact data only in `Member`, trainer contact data only in `Trainer`, and payments reference a membership rather than copying member or plan details.
- **Controlled redundancy:** `Membership.end_date` can be derived from `start_date` and the plan duration. It is stored deliberately so that expiry queries and reports stay simple, and the interface calculates it from the selected plan.

6.8 MongoDB Activity Logging

The `activity_logs` collection in `gym_membership_db` stores operational events that do not need relational structure. Its purpose is to keep a lightweight audit trail of activity alongside the relational data. Inspected documents contain `member_id`, `activity` (for example “Check-in”), `timestamp` and `details`, where `details` includes the attendance ID.

Table 6.13: MongoDB verification observed on the local service

Check	Observation
Service	Local MongoDB service running; `ping` returned `{ ok: 1 }`.
Database and collection	`gym_membership_db` contained the `activity_logs` collection.
Document count	6 documents at the time of inspection (observed, not forced).
Latest document	A Check-in event for member_id 11 with Attendance ID 14 in `details`.
Coverage	Verified for attendance only; trainer, assignment and payment actions were not shown to create documents.

7. Application Modules and Functionalities

This chapter describes each module of FITZONE as evidenced by the application screenshots, database inspection and project report. Where route-level details were not part of the supplied evidence, the profile says so rather than assuming behaviour. Figures show the live interface at the time of capture.

7.1 Dashboard

Table 7.1: Dashboard module profile

Aspect	Description
Purpose	Landing page giving an at-a-glance operational summary and navigation to every module.
Main functions	Summary cards for Total Members, Active Memberships, Memberships Expiring Soon (within 7 days), Total Trainers, Completed Payments and Pending Payments; a Member Directory with plan and status; Quick Navigation cards; a “Live Database Data” indicator.
User interaction	Search the member directory, open a module from the sidebar or quick-navigation cards, log out.
Backend / API	Figures are loaded server-side from the database. Dashboard-specific route handlers were not itemised in the supplied tree.
Database interaction	Counts and totals drawn from Member, Membership, Trainer and Payment data.
Validation / errors	Read-only page; no data entry.

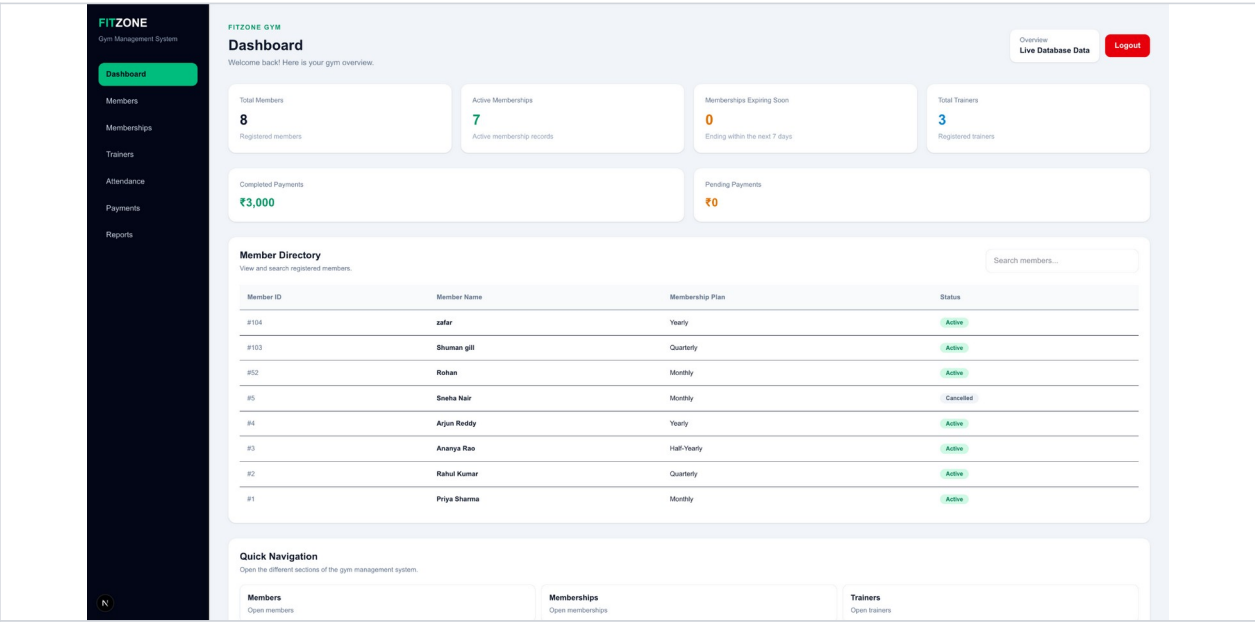


Figure 7.1: *Administrator dashboard*

7.2 Member Management

Table 7.2: Member management module profile

Aspect	Description
Purpose	Register gym members and look them up.
Main functions	Add New Member form (Full Name, Email, Phone, Gender) and a Member Directory (ID, Name, Email, Phone, Gender) with the number of members displayed.
User interaction	Fill in the form and choose “+ Add Member”; search the directory by name or email.
Backend / API	Member routes under `app/api/`; method-level detail not itemised in the supplied tree.
Database interaction	Inserts into and reads from `Member`; `join_date` and `created_at` take database defaults.
Validation / errors	NOT NULL on name, email and phone; UNIQUE on email and phone, so duplicates are rejected by the database.

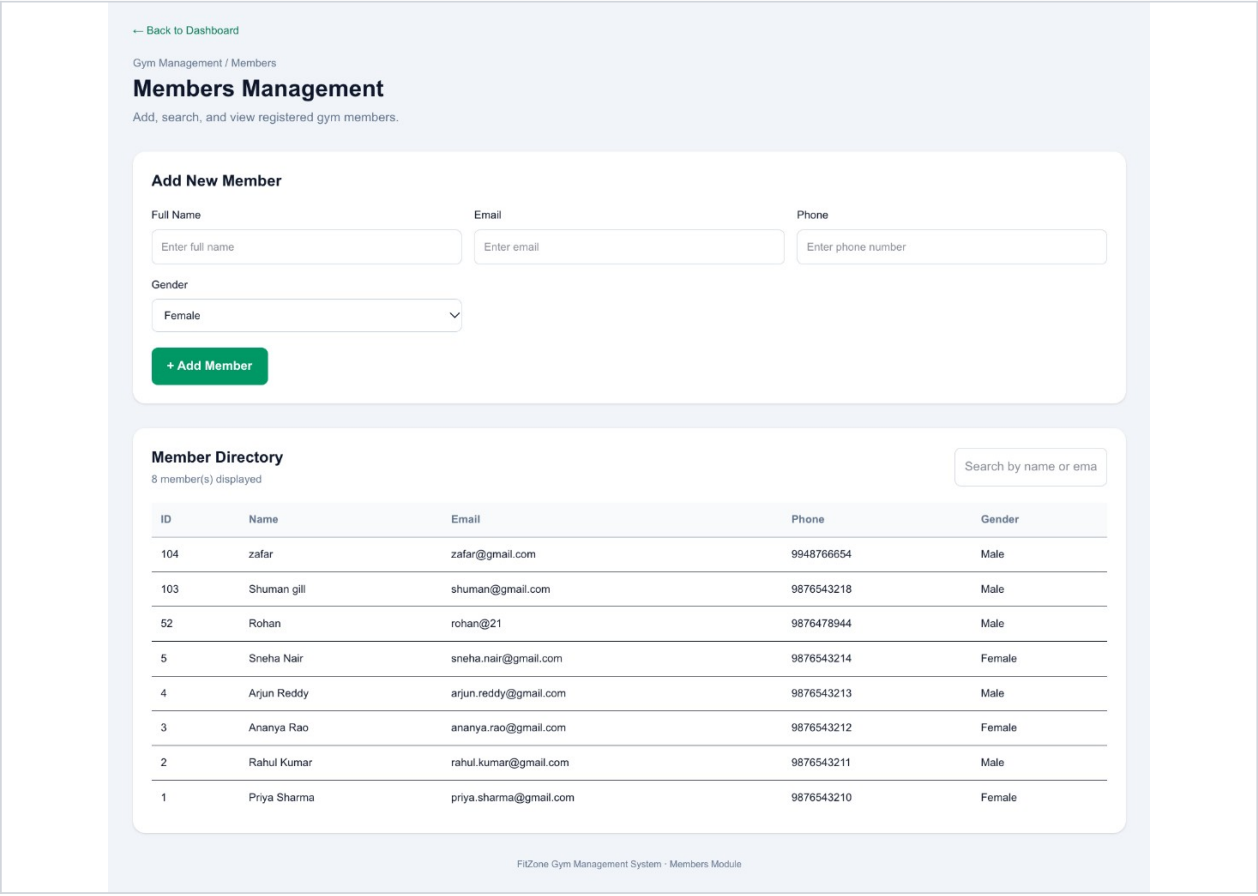


Figure 7.2: *Members Management page*

Observation. One directory entry has an email value without a domain (“rohan@21”). This suggests that email-format validation is not enforced on every entry path, or that the row was inserted directly. It should be checked and, if confirmed, listed as a limitation.

7.3 Membership Plan Management

Table 7.3: Membership plan module profile

Aspect	Description
Purpose	Define the plans a gym offers, with duration and price.
Main functions	Plans are stored in `Membership_Plan` and offered in the plan selector of the Membership page: Monthly, Quarterly, Half-Yearly and Yearly.
User interaction	The administrator chooses a plan when assigning a membership. A separate screen for creating or editing plans is not evidenced.
Backend / API	Plan data is read for the selector and membership listing; route detail not itemised.
Database interaction	`Membership_Plan` (plan_id, plan_name, duration_months, price, description).
Validation / errors	plan_name is unique; duration_months and price are NOT NULL; price is DECIMAL(10,2).

7.4 Membership Management

Table 7.4: Membership management module profile

Aspect	Description
Purpose	Assign plans to members and track membership periods and status.
Main functions	Assign a Membership form (Select Member, Membership Plan, Start Date, read-only Calculated End Date) and an All Memberships table (ID, Member, Plan with duration, Price, Start, End, Status) with a Refresh button and record count.
User interaction	Choose a member and a plan, set the start date, assign; change the status from the status control in the table.
Backend / API	Membership routes under `app/api/`; method-level detail not itemised.
Database interaction	Writes to and reads from `Membership`, joined to `Member` and `Membership_Plan`.
Validation / errors	Foreign keys on member and plan; status limited to Active, Expired or Cancelled; end date derived from the plan duration.

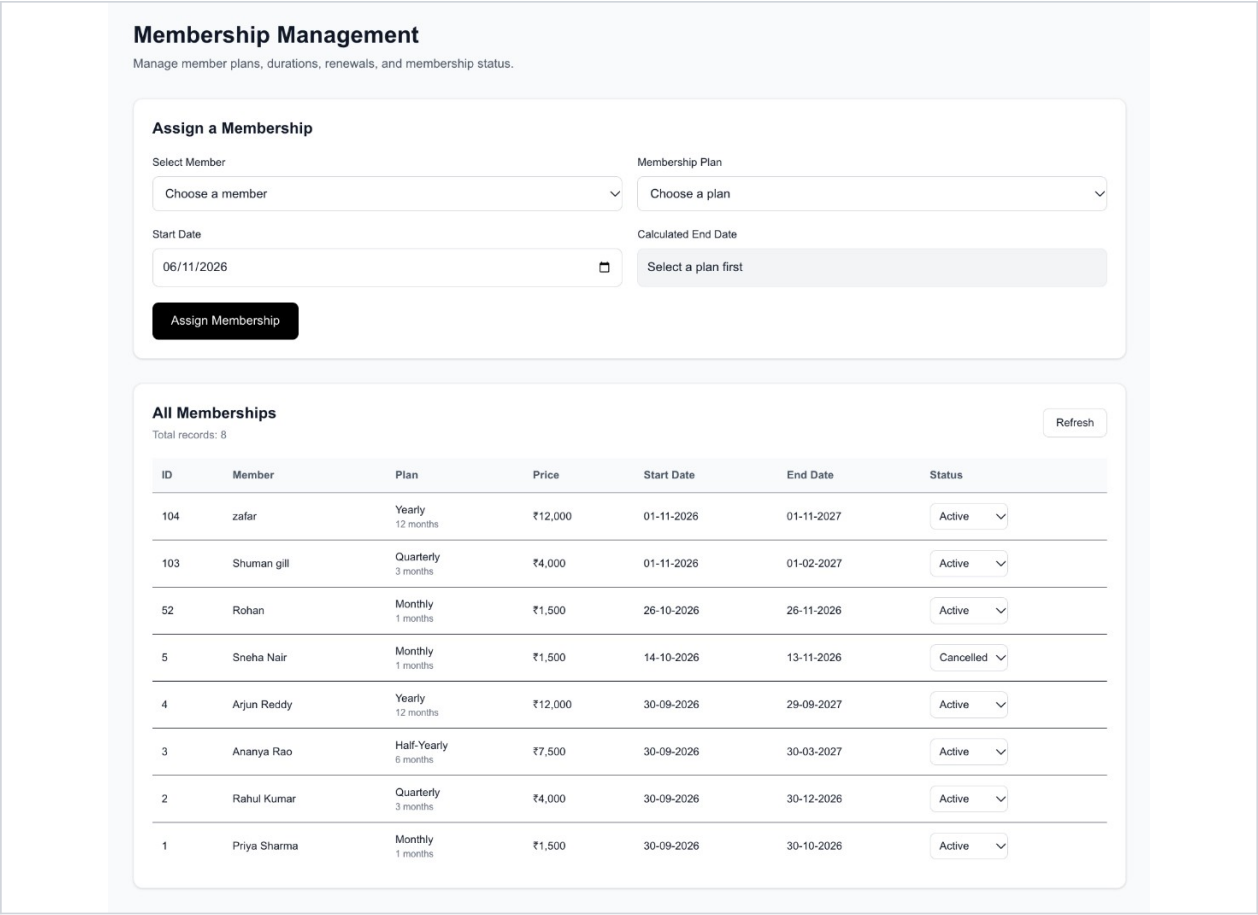


Figure 7.3: *Membership Management page*

7.5 Attendance Management

Table 7.5: Attendance module profile

Aspect	Description
Purpose	Record member check-ins and optional check-outs.
Main functions	Record Attendance form (Member, Attendance date, Check-in time, optional Check-out time) and an Attendance Records table (Member, Date, Check-in, Check-out, Status) with search by member or date.
User interaction	Select a member, enter date and times, choose “+ Record Attendance”; records appear with a “Present” status and “—” when no check-out was given.
Backend / API	`app/api/attendance`: validates the submitted values, inserts the attendance row in MySQL, then attempts to write a Check-in document to MongoDB.
Database interaction	`Attendance` (FK to `Member`) in MySQL; `activity_logs` in MongoDB.
Validation / errors	API validation plus the CHECK constraint on check-out time. MongoDB logging errors are handled separately from the MySQL insert.

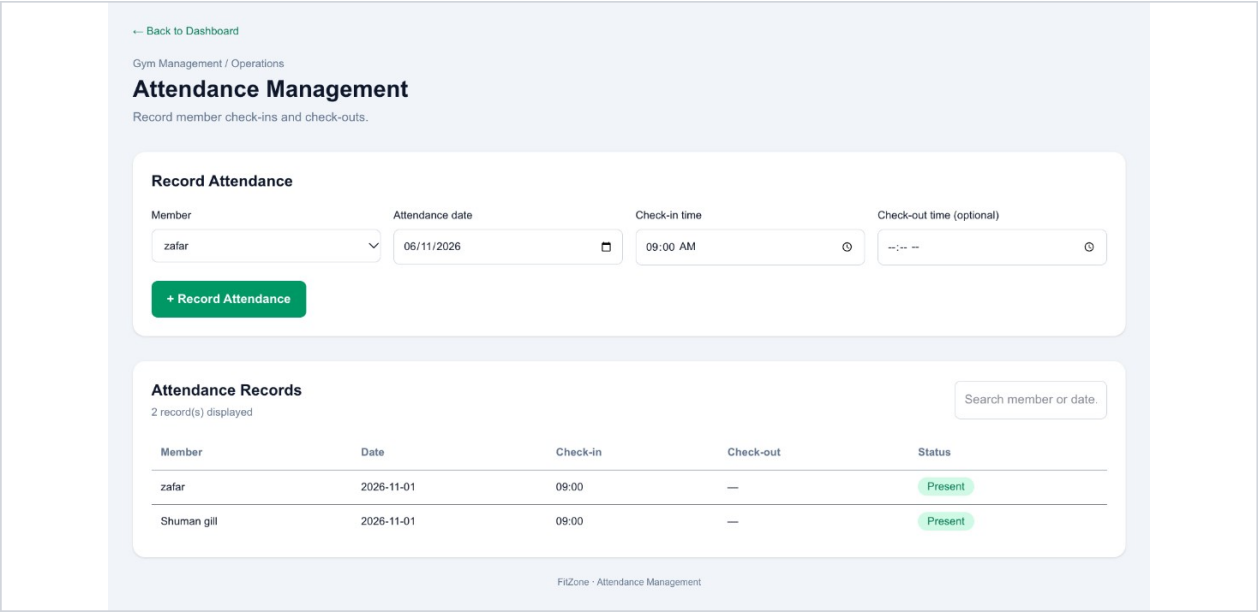


Figure 7.4: *Attendance Management page*

7.6 Trainer Management

Table 7.6: Trainer management module profile

Aspect	Description
Purpose	Maintain the trainer directory.
Main functions	Summary cards (Total Trainers, Active Trainers, Assignments); Add a Trainer form (name, specialization, phone, email); Trainer Directory with specialization, contact details, assigned-member count and status; search.
User interaction	Enter trainer details and choose “+ Add Trainer”; search the directory.
Backend / API	Trainer data operations run through route handlers; the exact route was not itemised in the supplied tree.
Database interaction	`Trainer`; assigned-member counts come from `Trainer_Assignment`.
Validation / errors	trainer_name is required; phone and email are unique. The `Trainer` table has no status column, so the “Active” label is presented by the application and should be confirmed as derived rather than stored.

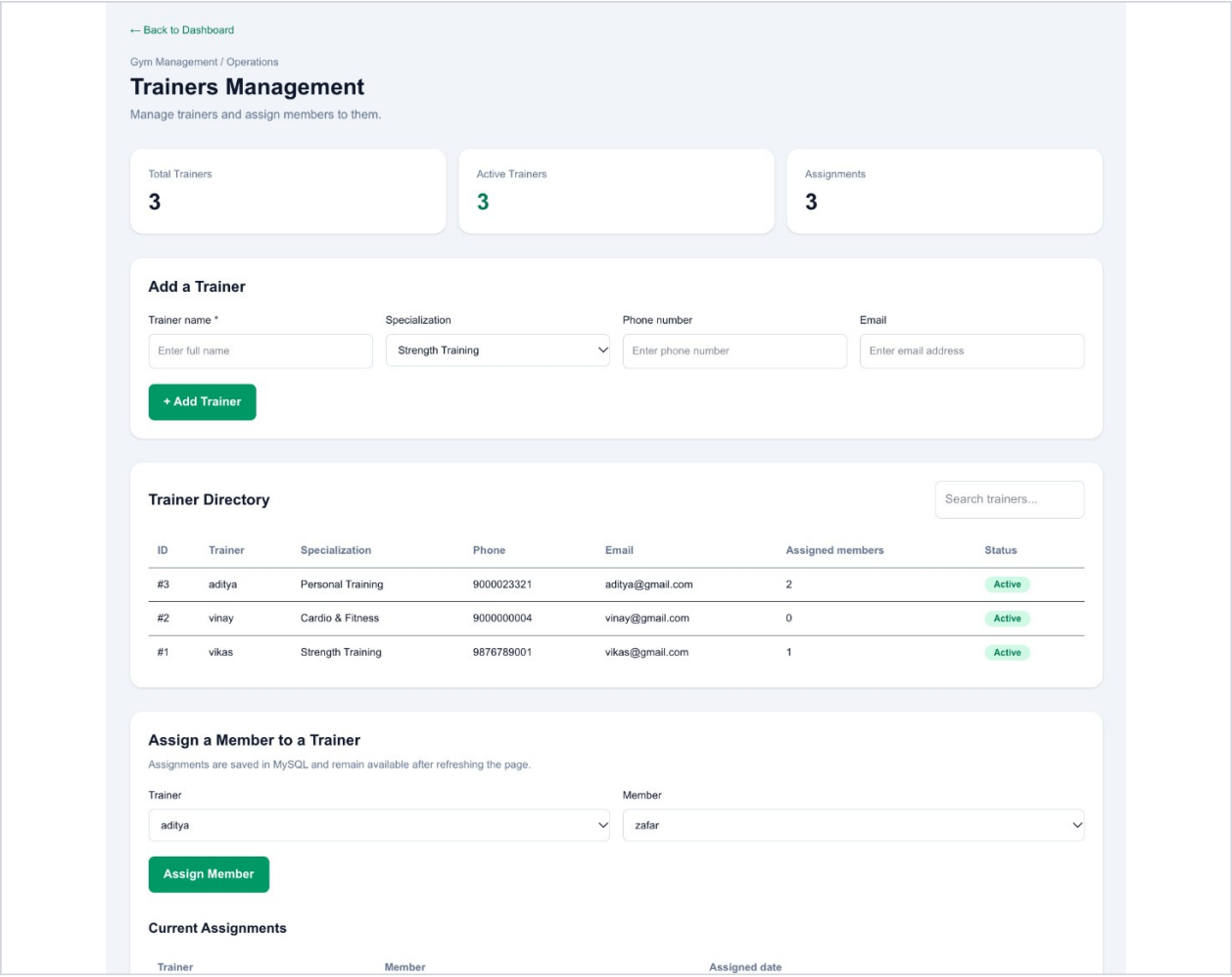


Figure 7.5: *Trainers Management page, including the assignment section*

7.7 Trainer Assignment Management

Table 7.7: Trainer assignment module profile

Aspect	Description
Purpose	Link members to trainers and show current assignments.
Main functions	Assign a Member to a Trainer (trainer and member selectors, “Assign Member”) and a Current Assignments list (Figure 7.5). The page states that assignments are saved in MySQL and remain available after refreshing.
User interaction	Choose a trainer and a member, then assign.
Backend / API	`app/api/assignments`, with a corresponding page under `app/assignments/`.
Database interaction	`Trainer_Assignment` (assigned_date, optional end_date) with foreign keys to `Trainer` and `Member`.
Validation / errors	Foreign keys require an existing trainer and member. Any rule preventing duplicate active assignments was not documented.

7.8 Payment Management

Table 7.8: Payment module profile

Aspect	Description
Purpose	Record payments against memberships and review payment history.
Main functions	Summary cards (Completed Revenue, Pending Payments); Record a Payment form (Membership/Member, Amount, Payment date, Method, Status); Payment History with search.
User interaction	Select a membership, enter amount, date, method and status, choose “+ Record Payment”.
Backend / API	`app/api/payments`.
Database interaction	`Payment` (FK to `Membership`); `GetMemberPayments` returns payments per member.
Validation / errors	CHECK amount >= 0; CHECK status in Pending, Completed, Failed, Refunded.

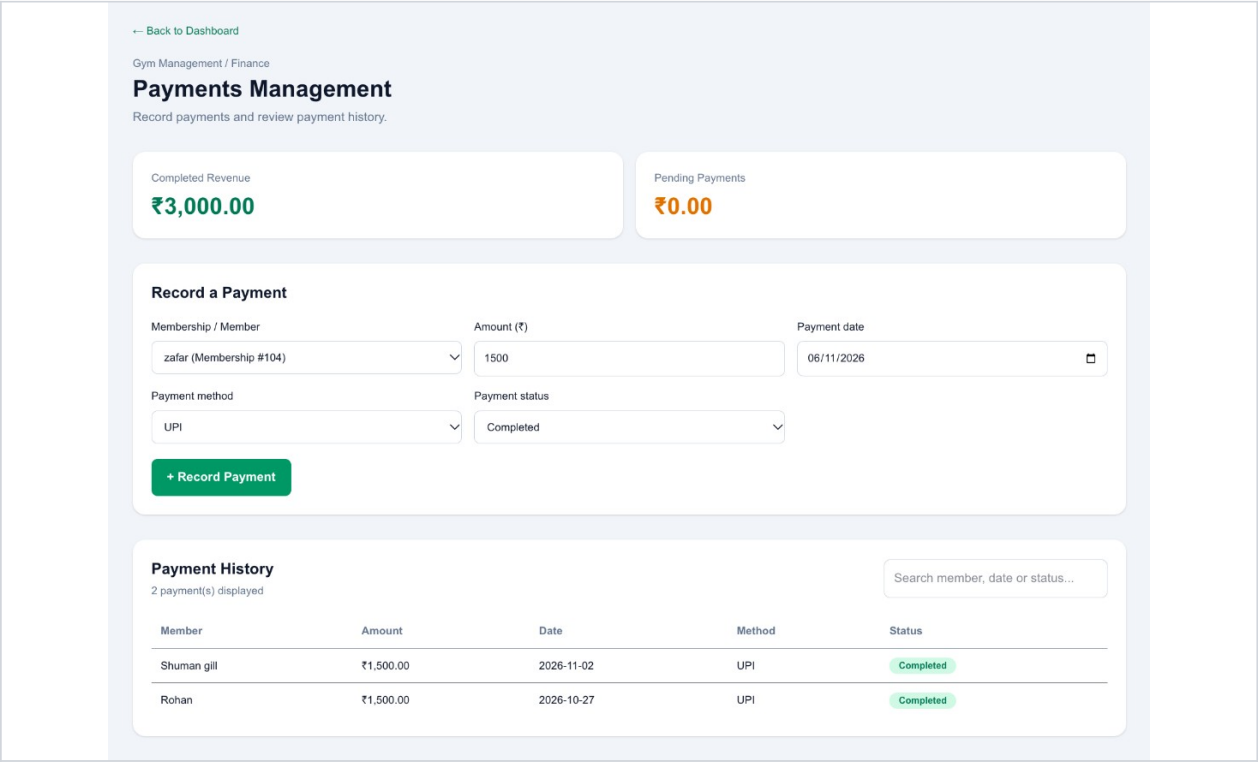


Figure 7.6: *Payments Management page*

7.9 Reports and Analytics

Table 7.9: Reports module profile

Aspect	Description
Purpose	Give administrators a consolidated view of attendance and payments.
Main functions	Summary cards (Total Members, Total Trainers, Completed Payments, Pending Payments); Attendance Report by date with Total Check-ins; Payment Report with an “All Payments” status filter; a “Live database records” footer.
User interaction	Open the page, review totals, filter payments by status.
Backend / API	Read-only report queries; routes not itemised in the supplied tree.
Database interaction	Reporting views `daily_attendance_report` and `payment_report` exist in the database; direct use by the page is not confirmed.
Validation / errors	Read-only; no data entry.

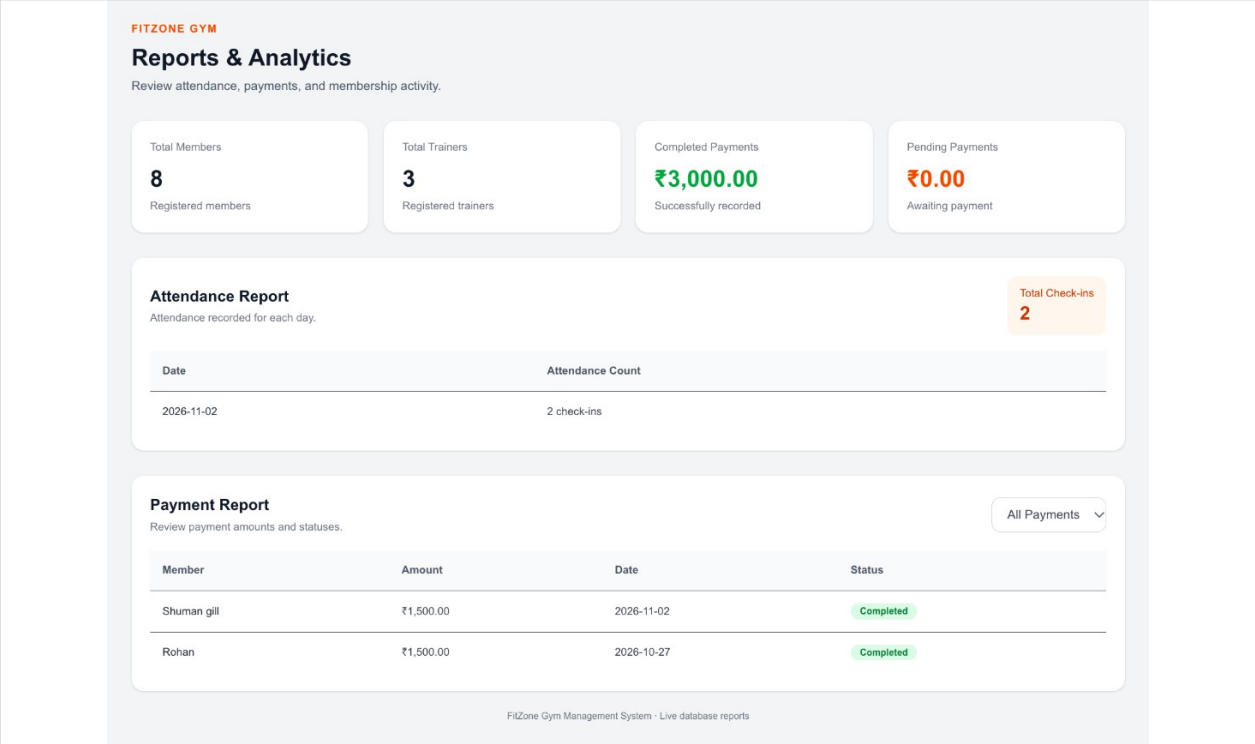


Figure 7.7: *Reports and Analytics page*

7.10 Authentication and User Management

Table 7.10: Authentication module profile

Aspect	Description
Purpose	Control access and connect a login to an administrator role or to a member.
Main functions	Email and password sign-in; password check against a bcrypt hash; signed session token; Logout button; Member Dashboard for member accounts.
User interaction	Sign in, work within the dashboard, log out.
Backend / API	Uses `bcryptjs` and `jose`; `AUTH_SECRET` read from `.env.local`.
Database interaction	`auth_users` (user_id, full_name, email, password_hash, role, member_id, created_at).
Validation / errors	Unique email and member_id; foreign key to Member with ON DELETE SET NULL. A self-registration screen is not evidenced.

Local versus shared setup. The local administrator account was created through `auth_users_setup.sql` and verified by logging in. The other team member used a separate local database. The local script must be compared with the shared authentication setup before merging or deploying. A test member email was absent from the local Member table, so member-account authentication remains pending.

7.11 Activity Logging

Table 7.11: Activity logging module profile

Aspect	Description
Purpose	Keep a lightweight event trail in MongoDB.
Main functions	Check-in events recorded when attendance is saved; documents hold member_id, activity, timestamp and details.
User interaction	None directly; events are created as a side effect of recording attendance.
Backend / API	Written from the attendance route through `lib/mongodb.ts`; a route directory `app/api/activity-logs/` also exists, but its methods were not itemised.
Database interaction	MongoDB collection `activity_logs`.
Validation / errors	Errors are handled separately from the MySQL insert. Logging of trainer, assignment and payment actions is not established.

8. API and Backend Implementation

8.1 Overview

The backend is implemented with Next.js Route Handlers in `app/api/`. Handlers use `lib/db.ts` for MySQL and `lib/mongodb.ts` for MongoDB. Table 8.1 lists the route directories identified in the project tree and what is documented about each.

Table 8.1: API route directories and documented responsibilities

Route directory	Module	Documented responsibility	Verification
`app/api/attendance`	Attendance	Validates submitted attendance values; inserts the attendance row in MySQL; attempts a Check-in document in MongoDB.	Manually tested; MongoDB log observed
`app/api/assignments`	Trainer assignment	Stores and returns trainer–member assignments in MySQL.	Manually tested
`app/api/payments`	Payments	Records and returns payments in MySQL; amount and status protected by CHECK constraints.	Manually tested
`app/api/activity-logs`	Activity logging	Route directory for activity-log data in MongoDB.	Not separately tested
Other routes under `app/api/`	Dashboard, members, memberships, trainers, reports, authentication	Present in the application (listed as “other pages and API routes”); not itemised in the supplied tree.	To be documented

Scope of this chapter. HTTP methods, request bodies and response codes for each handler are not reproduced because the route source files were not part of the supplied evidence. No endpoint or behaviour has been invented. Appendix C records the data stores each route is expected to use, and the checklist in Appendix D lists what should be added from the source.

8.2 Request and Response Handling

Pages send requests to the route handlers when a form is submitted or a list is loaded, and render the returned data in tables and cards. The attendance flow is the best documented: the handler receives the submitted member, date and times, validates them, writes to MySQL and then attempts the MongoDB log entry.

8.3 Database Operations

- **MySQL:** parameterised access through `mysql2` for inserts, selects and joins across the core tables; reporting views and `GetMemberPayments` for reporting.
- **MongoDB:** insertion of Check-in documents into `activity_logs` through the `mongodb` driver.

8.4 Validation and Error Handling

- The attendance API validates submitted values before inserting.

- The database rejects rows that violate unique, foreign-key, ENUM or CHECK rules.
- MongoDB logging errors are handled separately so that a logging failure does not mask the outcome of the relational write.

8.5 Authentication and Authorisation

Credential checks use `bcryptjs` against `password_hash` in `auth_users`, and a session token is signed with `jose` using `AUTH_SECRET`. The supplied evidence does not state how individual routes verify the token or enforce the administrator and member roles; this should be confirmed from the source.

8.6 Frontend–Backend Integration

Each page under `app/` corresponds to a module and talks to a matching route directory under `app/api/` where one exists. Integration of the Member 2 modules with the shared application was carried out on branch `member2/integration` and verified by a local production build and by GitHub Actions (Chapter 10).

9. User Interface and Application Workflow

This chapter describes the workflows an administrator follows. Screenshots of each module are in Chapter 7; the figures here show navigation and the member view.

9.1 Login and Registration

1. Open the application and enter the account email and password.
2. The credentials are checked against the stored hash and a signed session is created.
3. An administrator is taken to the dashboard; a member account is taken to the Member Dashboard.

Admin login to the dashboard was verified locally. A self-registration screen is not part of the evidenced implementation; accounts are created through the setup script.

9.2 Dashboard Navigation

The dark sidebar lists Dashboard, Members, Memberships, Trainers, Attendance, Payments and Reports, with the active item highlighted in green and the signed-in administrator shown at the bottom. The main area repeats these destinations as Quick Navigation cards. Assignments are managed from the Trainers page.

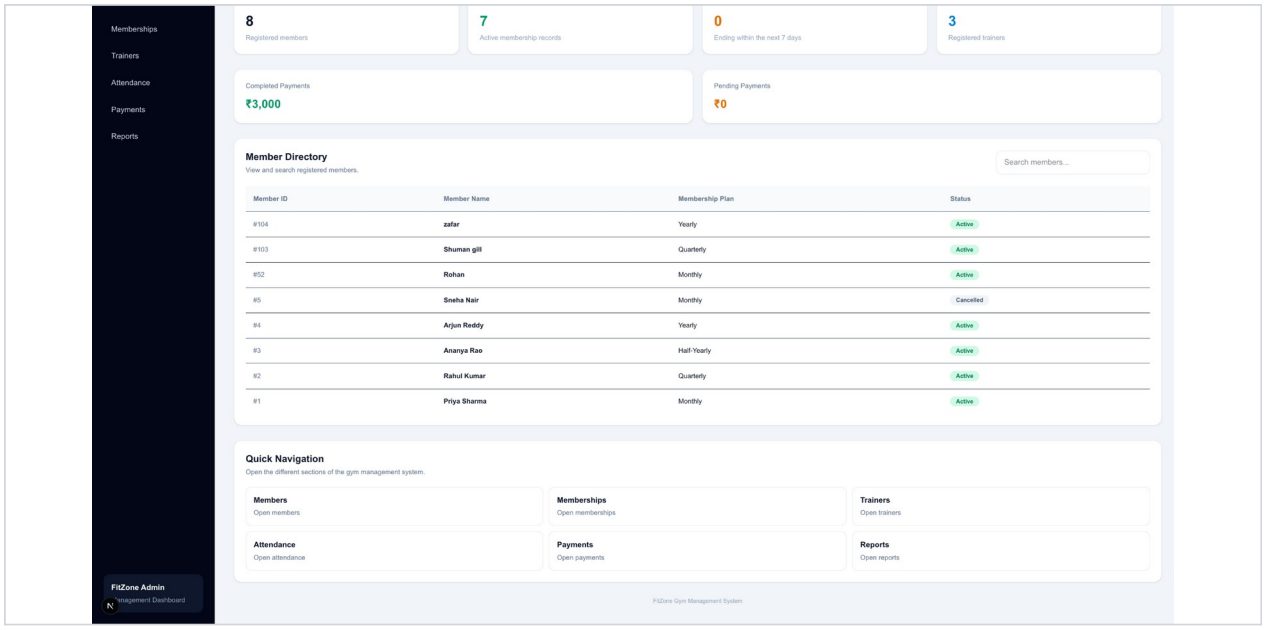


Figure 9.1: *Dashboard sidebar and Quick Navigation cards*

9.3 Member Management Workflow

1. Open **Members** and complete Full Name, Email, Phone and Gender.
2. Choose **+ Add Member**; the new member appears in the Member Directory.
3. Use the search box to find members by name or email.

9.4 Membership Workflow

1. Open **Memberships** and choose a member and a plan.
2. Select the start date; the end date is calculated and shown read-only.
3. Choose **Assign Membership**; the record appears in All Memberships.
4. Change the status from the status control when a membership expires or is cancelled.

9.5 Attendance Check-in and Check-out

1. Open **Attendance** and select the member and date.
2. Enter the check-in time and, if already known, the check-out time.
3. Choose **+ Record Attendance**. The record is listed with a Present status; a missing check-out shows as a dash.
4. For a check-in, a Check-in activity document is also attempted in MongoDB.

> **Note.** The supplied evidence shows check-out entered on the same form as check-in. How a check-out is added to an existing record later is not documented.

9.6 Trainer Assignment Workflow

1. Open **Trainers** and add trainer details if the trainer is not yet listed.
2. In **Assign a Member to a Trainer**, choose the trainer and the member.
3. Choose **Assign Member**; the assignment appears under Current Assignments and the trainer's assigned-member count updates.

9.7 Payment Workflow

1. Open **Payments** and choose the membership or member.
2. Enter the amount and payment date and choose the method and status.
3. Choose **+ Record Payment**; the entry appears in Payment History and the revenue cards update.

9.8 Reports Generation

1. Open **Reports** to see the summary cards.
2. Read the daily attendance counts in the Attendance Report.
3. Use the status filter in the Payment Report to focus on completed, pending or other payments.

9.9 Member Self-Service View

A member account opens the Member Dashboard, which shows the member ID, plan and membership status, followed by My Memberships, My Attendance and My Payment History.

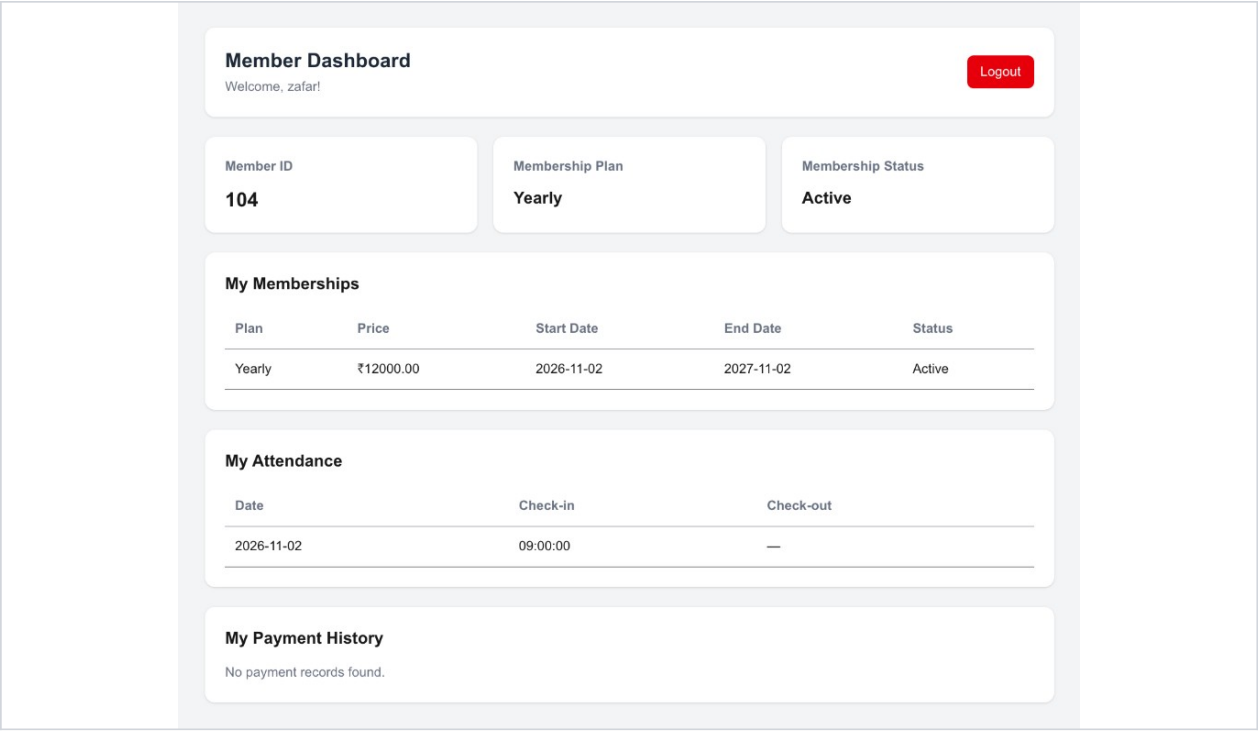


Figure 9.2: *Member Dashboard*

10. Testing and Validation

10.1 Testing Approach and Evidence Policy

Results in this chapter are limited to what is supported by evidence. Each row carries one of four statuses.

Table 10.1: Status legend

Status	Meaning
Passed	Supported by a recorded observation, build result or inspection output.
Manual	Tested manually by a team member; no detailed test log was supplied.
Pending	Not yet executed or not yet evidenced.
Open	An issue or unresolved check was identified.

10.2 Functional Testing

Table 10.2: Functional test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-F01	Attendance	Record attendance with member, date and check-in time.	Record saved and listed with Present status.	Page and action tested manually; records appear in the list.	**Manual**
TC-F02	Attendance	Record attendance without a check-out time.	Record accepted; check-out shown as a dash.	Records without check-out are displayed with a dash.	**Manual**
TC-F03	Trainers	Add a trainer with name, specialization, phone, email.	Trainer appears in the Trainer Directory.	Page and actions checked manually.	**Manual**
TC-F04	Assignments	Assign a member to a trainer.	Assignment listed and stored in MySQL.	Checked manually; assigned-member counts shown in the directory.	**Manual**
TC-F05	Payments	Record a completed payment.	Payment in history; revenue card updated.	Checked manually; completed revenue shows the recorded total.	**Manual**
TC-F06	Reports	Open Reports and review totals.	Summary cards, attendance and payment reports render.	Section opened and checked by the contributor.	**Manual**

10.3 API Testing

Table 10.3: API test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-A01	Attendance API	Create an attendance record through the UI.	Row inserted in MySQL and Check-in document attempted in MongoDB.	Attendance UI tested; Check-in document observed in MongoDB.	Manual
TC-A02	Attendance API	Submit invalid values (check-out before check-in).	Request rejected with an error.	Validation exists in the API; negative test not recorded.	Pending
TC-A03	All routes	Automated API test suite.	Requests and responses verified per route.	No automated API tests were recorded.	Pending

10.4 Database Testing

Table 10.4: Database test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-D01	Schema	Run SHOW TABLES on gym_membership_db.	Core tables and reporting objects listed.	Nine objects listed (see Appendix B).	Passed
TC-D02	Schema	DESCRIBE Member, Membership_Plan, Membership, Trainer.	Keys, unique keys, types and defaults shown.	Output captured and consistent with the design.	Passed
TC-D03	Views and procedure	Inspect reporting views and GetMemberPayments.	Objects present and queryable.	Tables, views and procedure inspected locally.	Passed
TC-D04	Views	List objects with SHOW FULL TABLES.	daily_attendance_report and payment_report listed as views.	These two names are absent from the supplied SHOW TABLES output.	Open

10.5 Integration Testing

Table 10.5: Integration test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-I01	Authentication	Sign in with the local administrator account.	Dashboard opens.	Login succeeded and the dashboard opened.	Passed
TC-I02	Authentication	Sign in with a member account.	Member Dashboard opens.	The provided test email was absent from the local Member table.	Pending
TC-I03	Attendance + MongoDB	Record attendance and inspect activity_logs.	A Check-in document exists.	Check-in documents observed; latest referenced Attendance ID 14.	Passed
TC-I04	Logging coverage	Trainer, assignment and payment actions create MongoDB documents.	Documents created (if required).	Inspected code did not establish this; acceptance criterion to be confirmed.	Open
TC-I05	Shared authentication	Compare auth_users script with the teammate's version.	Schemas consistent.	Not yet compared; teammate used a separate local database.	Pending

10.6 Manual Testing

The contributor manually tested the five Member 2 sections (Attendance, Trainers, Trainer Assignments, Payments and Reports) and their actions in the running application. A detailed manual test log was not supplied; the functional rows above summarise the reported outcome. The test checks on MongoDB are listed below.

Table 10.6: MongoDB checks

ID	Module	Test scenario	Expected result	Actual result	Status
TC-M01	MongoDB	Run ping against the local service.	ok: 1	`{ ok: 1 }` returned.	Passed
TC-M02	MongoDB	List collections in gym_membership_db.	activity_logs present.	Collection present.	Passed
TC-M03	MongoDB	Count documents in activity_logs.	Count reported.	6 documents (observed, not forced).	Passed

10.7 Build Verification and GitHub Actions Checks

Table 10.7: Build and CI test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-B01	Build	Run the Next.js production build locally.	Compile and TypeScript checks pass.	Passed; 30 static pages generated.	Passed
TC-B02	CI	GitHub Actions “Build Check” workflow.	Workflow succeeds.	Two successful Build Check runs reported.	Passed
TC-B03	Lint	Run ESLint across the repository.	No errors.	Earlier repository-wide run reported remaining issues in other or shared files.	Open

10.8 Validation and Error-Handling Tests

The constraints below are present in the schema. Negative tests that deliberately violate them were not recorded, so they are listed as pending rather than as passed.

Table 10.8: Validation test cases

ID	Module	Test scenario	Expected result	Actual result	Status
TC-V01	Attendance	Insert a check-out earlier than the check-in.	Rejected by CHECK constraint or API.	Constraint present; test not recorded.	Pending
TC-V02	Payments	Insert a negative amount.	Rejected by CHECK (amount >= 0).	Constraint present; test not recorded.	Pending
TC-V03	Payments	Insert a status outside the allowed set.	Rejected by CHECK on status.	Constraint present; test not recorded.	Pending
TC-V04	Members	Insert a duplicate email or phone.	Rejected by UNIQUE key.	Unique keys present; test not recorded.	Pending
TC-V05	Assignments	Assign a non-existent trainer or member.	Rejected by foreign key.	Keys present; test not recorded.	Pending
TC-V06	Attendance / Reports	Compare displayed dates of the same check-ins on the Attendance, Reports and Member Dashboard pages.	Same date on every page.	Screenshots show 2026-11-01 on Attendance and 2026-11-02 on Reports and Member Dashboard; cause to be investigated.	Open

10.9 Summary of Test Status

Of 30 recorded test cases, 10 are **Passed**, 7 are **Manual**, 9 are **Pending** and 4 are **Open**. Completed tests (Passed and Manual) cover schema inspection, MongoDB connectivity, administrator login, attendance logging, the five Member 2 sections, the local build and CI. Pending tests are mainly negative tests of constraints, member-account login and shared-authentication alignment.

Table 10.9: Test status summary

Status	Count
Passed	10
Manual	7
Pending	9
Open	4
Total	30

11. Version Control and Team Collaboration

11.1 Git and GitHub Workflow

The project is hosted in the GitHub repository `priyankabm11/Gym-Membership-Management-System`. Work on the Member 2 modules and integration was committed on the branch `member2/integration` and pushed to GitHub, from which Pull Request #1 was opened.

11.2 Repository Structure

The following focused tree is taken from the project report. It is not a complete listing.

```
Gym-Membership-Management-System/
|-- app/
|   |-- attendance/
|   |-- trainers/
|   |-- assignments/
|   |-- payments/
|   |-- reports/
|   |-- api/
|   |   |-- attendance/
|   |   |-- assignments/
|   |   |-- payments/
|   |   `-- activity-logs/
|   `-- ... (other pages and API routes)
|-- lib/
|   |-- db.ts
|   `-- mongodb.ts
|-- .github/workflows/build.yml
|-- auth_users_setup.sql
|-- member2_integration.sql
|-- package.json
`-- package-lock.json
```

11.3 Branching Strategy

Member 2's work was developed on a dedicated feature branch, `member2/integration`, separate from the main line of development. Any wider branching convention used by the team is not documented in the supplied materials.

11.4 Pull Request Workflow

Pull Request #1 (`https://github.com/priyankabm11/Gym-Membership-Management-System/pull/1`) is **open**. Its review and merge status is still to be confirmed, so this report does not describe it as merged.

11.5 Code Review

No completed review or approval was recorded in the supplied evidence. Review of the integration script against the target schema, and comparison of the authentication scripts, are part of the outstanding integration work.

11.6 Continuous Integration

The workflow `.github/workflows/build.yml` runs a “Build Check” that installs dependencies with npm and builds the application. Two successful Build Check runs were reported. A successful build confirms that the code compiles and type-checks; it does not replace functional testing, and ESLint has not been reported clean.

11.7 Team Collaboration and Integration Process

Table 11.1: Team members and responsibilities

Member	SRN	Documented responsibility
Ramalakshmi M N (Member 2)	PES1UG24AM219	Database integration; application modules and integration fixes for Attendance, Trainers, Trainer Assignments, Payments and Reports; MongoDB checks; testing; documentation.
Priyanka B M (Member 1)	PES1UG24AM207	Responsibilities were not documented in the supplied materials. Modules outside the five Member 2 sections (for example dashboard, members, memberships and the shared authentication setup) were developed outside Member 2’s documented scope.

11.8 Member 2 Contribution within the Overall Project

The Member 2 contribution forms part of the overall system rather than a separate product. It covers the Attendance, Trainer, Trainer Assignment, Payment and Reports modules, the supporting MySQL objects (tables, views and `GetMemberPayments`, scripted in `member2_integration.sql`), the MongoDB attendance logging, the verification work reported in Chapter 10 and the project documentation. These modules share the Member and Membership data and the authentication layer with the rest of FITZONE.

11.9 Integration Considerations

- Compare the local `auth_users` setup with the teammate’s version and agree on a single shared schema.
- Review `member2_integration.sql` against the target database before running it, because `CREATE TABLE IF NOT EXISTS` does not retrofit constraints onto existing tables.
- Keep `.env.local` out of version control and out of screenshots.
- Confirm PR #1 review and merge status.

12. Results and Discussion

12.1 Overall System Functionality

FITZONE provides a working administrative interface for the core gym operations: members, memberships, trainers and assignments, attendance, payments, reports and a member view, backed by a constrained MySQL schema and supplemented by MongoDB activity logging. The application builds successfully and the Member 2 modules were exercised manually.

12.2 Completed Modules

Table 12.1: Module status and evidence

Module	Status	Evidence
Dashboard	Implemented	Screenshot shows live summary cards and directory.
Member Management	Implemented	Screenshot shows add form and directory.
Membership Plans	Data present	Four plans visible in the Membership page; no standalone plan screen evidenced.
Membership Management	Implemented	Screenshot shows assignment form and membership table.
Attendance	Implemented; manually tested	Manual test; MongoDB Check-in log observed.
Trainers	Implemented; manually tested	Manual test; screenshot.
Trainer Assignments	Implemented; manually tested	Manual test; page states assignments persist in MySQL.
Payments	Implemented; manually tested	Manual test; constraints present.
Reports	Implemented; manually tested	Manual check; screenshot.
Admin authentication	Verified locally	Local auth_users row; admin login opened the dashboard.
Member authentication	Pending	Test email absent from local Member table.
Activity logging	Verified for attendance only	MongoDB inspection.

12.3 Integration Results

The Member 2 integration branch builds locally (compile and TypeScript checks passed, 30 static pages generated) and passed two GitHub Actions Build Check runs. Pull Request #1 remains open, and the authentication scripts have still to be reconciled with the teammate's database.

12.4 Testing Outcomes

Of 30 recorded test cases, 10 passed, 7 were completed manually, 9 are pending and 4 are open (Chapter 10). The open items are the missing reporting views in the supplied schema listing, MongoDB

logging coverage, ESLint issues in other or shared files and an apparent date difference between pages.

12.5 Benefits Achieved

- Gym records are centralised in one database with enforced relationships.
- Invalid payments and inconsistent attendance times are blocked at the database layer.
- Administrators get live totals, searchable lists and reports without manual compilation.
- Attendance activity is traceable in MongoDB.

12.6 Current Limitations

- Member-account authentication is not yet aligned with the shared Member data.
- MongoDB logging is verified only for attendance.
- ESLint has not been reported clean; issues remain in other or shared files.
- Negative tests for constraints and an automated test suite have not been recorded.
- Route-level authorisation rules are not documented in the supplied evidence.
- A sample member record has an email value without a domain, and some pages show different dates for the same check-ins; both need checking.
- Pull Request #1 is open and the final integration review is outstanding.
- The system has not been presented as production-ready; no deployment, load or security testing was recorded.

13. Future Enhancements

All items below are **proposed** enhancements. None is implemented.

Table 13.1: Proposed enhancements

Proposal	Rationale
Align member authentication with shared Member data and one shared `auth_users` schema	Completes member sign-in and removes the difference between local databases.
Extend MongoDB activity logging to trainer, assignment and payment actions	Gives a uniform audit trail, if the team confirms it as a requirement.
Add automated API and database tests, including negative constraint tests	Turns the pending validation checks into repeatable evidence.
Resolve ESLint findings and enforce linting in CI	Improves code quality beyond a successful build.
Add email-format and phone-format validation on every entry path	Prevents records such as an email value without a domain.
Review date and time-zone handling across pages	Removes the apparent one-day difference seen in screenshots.
Add a plan-management screen	Lets administrators create and edit plans without database access.
Review and test route-level role-based authorisation	Makes the administrator and member access levels explicit and verified.
Membership expiry reminders	Supports renewal, building on the “expiring within 7 days” dashboard card.

14. Conclusion

FITZONE is a gym operations management system built with Next.js, React and TypeScript on top of a MySQL 8.0 database and a MongoDB activity-log collection. The relational design links members, plans, memberships, payments, attendance, trainers and trainer assignments with primary keys, foreign keys and validation constraints, and the application exposes these through administrative pages for the dashboard, members, memberships, trainers, assignments, attendance, payments and reports, together with a personal Member Dashboard.

The Member 2 work completed the integration of the attendance, trainer, assignment, payment and reporting modules with the database, verified MongoDB logging for attendance, and confirmed that the application builds locally and in GitHub Actions. Administrator login was verified locally, and the five Member 2 sections were tested manually.

Several items remain before the system can be treated as fully integrated: aligning member authentication with the shared data, reviewing and merging Pull Request #1, recording negative constraint tests and resolving the open findings listed in Chapter 12. With these completed, FITZONE would provide a solid foundation for managing a gym’s members, memberships, attendance, trainers and payments in one place.

References

[1] Vercel, Inc., “Next.js Documentation,” https://nextjs.org/docs

[2] Meta Platforms, Inc., “React Documentation,” https://react.dev

[3] Microsoft, “TypeScript Documentation,” https://www.typescriptlang.org/docs/

[4] OpenJS Foundation, “Node.js Documentation,” https://nodejs.org/docs/latest/api/

[5] Oracle Corporation, “MySQL 8.0 Reference Manual,” https://dev.mysql.com/doc/refman/8.0/en/

[6] MongoDB, Inc., “MongoDB Manual,” https://www.mongodb.com/docs/manual/

[7] MongoDB, Inc., “MongoDB Node.js Driver,” https://www.mongodb.com/docs/drivers/node/current/

[8] S. Sidorares et al., “node-mysql2,” https://github.com/sidorares/node-mysql2

[9] bcrypt.js contributors, “bcrypt.js,” https://github.com/dcodeIO/bcrypt.js

[10] P. Dolezal, “jose – JavaScript module for JSON Object Signing and Encryption,” https://github.com/panva/jose

[11] GitHub, Inc., “GitHub Actions Documentation,” https://docs.github.com/actions

[12] ESLint, “ESLint Documentation,” https://eslint.org/docs/latest/

[13] FITZONE project repository, https://github.com/priyankabm11/Gym-Membership-Management-System

[14] FITZONE Pull Request #1, https://github.com/priyankabm11/Gym-Membership-Management-System/pull/1

[15] FITZONE project artefacts: `member2_integration.sql`, `auth_users_setup.sql`, `lib/db.ts`, `lib/mongodb.ts`, and the application and database screenshots supplied by the project team.

Appendix A: Database Scripts and Queries

The statements below are those documented in the project report. Full scripts are in the repository files `member2_integration.sql` and `auth_users_setup.sql`.

A.1 MySQL Statements

A.1.1 Schema inspection

```
USE gym_membership_db;
SHOW TABLES;
DESCRIBE attendance;
DESCRIBE trainer;
DESCRIBE trainer_assignment;
DESCRIBE payment;
```

A.1.2 Validation constraints

```
-- Attendance
CHECK (check_out_time IS NULL OR check_out_time >= check_in_time)

-- Payment
CHECK (amount >= 0)
CHECK (status IN ('Pending', 'Completed', 'Failed', 'Refunded'))
```

A.1.3 Reports and stored procedure

```
SELECT * FROM daily_attendance_report;
SELECT * FROM payment_report;
CALL GetMemberPayments(1);
```

A.2 MongoDB Shell Commands

```
mongosh "mongodb://127.0.0.1:27017/gym_membership_db"
db.getCollectionNames()
db.activity_logs.countDocuments()
db.activity_logs.find().sort({_id:-1}).limit(5).pretty()
```

A.3 Authentication Table Specification

`auth_users_setup.sql` creates `auth_users` with `user_id`, `full_name`, `email`, `password_hash`, `role`, `member_id` and `created_at`. It defines unique keys on `email` and `member_id` and a foreign key from `member_id` to `Member(member_id)` with ON DELETE SET NULL. The local administrator account was created with a bcrypt hash and verified by a successful login. The script is local and should be compared with the team's shared setup before merging or deploying.

Appendix B: Supporting Evidence

B.1 Database Inspection Screenshots

```
mysql> USE gym_membership_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+------------------------------+
| Tables_in_gym_membership_db |
+------------------------------+
| attendance                   |
| auth_users                   |
| Member                       |
| member_membership_report     |
| Membership                   |
| Membership_Plan              |
| payment                      |
| trainer                      |
| trainer_assignment           |
+------------------------------+
9 rows in set (0.001 sec)

mysql>
```

Figure B.1: *SHOW TABLES output for gym_membership_db*

```
mysql> DESCRIBE Member;
+-------------+--------------+------+-----+-------------------+-------------------+
| Field       | Type         | Null | Key | Default           | Extra             |
+-------------+--------------+------+-----+-------------------+-------------------+
| member_id   | int          | NO   | PRI | NULL              | auto_increment    |
| full_name   | varchar(100) | NO   |     | NULL              |                   |
| email       | varchar(100) | NO   | UNI | NULL              |                   |
| phone       | varchar(15)  | NO   | UNI | NULL              |                   |
| gender      | varchar(20)  | YES  |     | NULL              |                   |
| join_date   | date         | NO   |     | curdate()         | DEFAULT_GENERATED |
| created_at  | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+--------------+------+-----+-------------------+-------------------+
7 rows in set (0.004 sec)

mysql> DESCRIBE Membership_Plan;
+-----------------+---------------+------+-----+---------+----------------+
| Field           | Type          | Null | Key | Default | Extra          |
+-----------------+---------------+------+-----+---------+----------------+
| plan_id         | int           | NO   | PRI | NULL    | auto_increment |
| plan_name       | varchar(50)   | NO   | UNI | NULL    |                |
| duration_months | int           | NO   |     | NULL    |                |
| price           | decimal(10,2) | NO   |     | NULL    |                |
| description     | varchar(255)  | YES  |     | NULL    |                |
+-----------------+---------------+------+-----+---------+----------------+
5 rows in set (0.001 sec)

mysql> DESCRIBE Membership;
+---------------+-----------------------------------------+------+-----+---------+----------------+
| Field         | Type                                    | Null | Key | Default | Extra          |
+---------------+-----------------------------------------+------+-----+---------+----------------+
| membership_id | int                                     | NO   | PRI | NULL    | auto_increment |
| member_id     | int                                     | NO   | MUL | NULL    |                |
| plan_id       | int                                     | NO   | MUL | NULL    |                |
| start_date    | date                                    | NO   |     | NULL    |                |
| end_date      | date                                    | NO   |     | NULL    |                |
| status        | enum('Active','Expired','Cancelled')    | NO   | MUL | Active  |                |
+---------------+-----------------------------------------+------+-----+---------+----------------+
6 rows in set (0.001 sec)

mysql> DESCRIBE Trainer;
+----------------+--------------+------+-----+---------+----------------+
| Field          | Type         | Null | Key | Default | Extra          |
+----------------+--------------+------+-----+---------+----------------+
| trainer_id     | int          | NO   | PRI | NULL    | auto_increment |
| trainer_name   | varchar(100) | NO   |     | NULL    |                |
| specialization | varchar(100) | YES  |     | NULL    |                |
| phone          | varchar(15)  | YES  | UNI | NULL    |                |
| email          | varchar(100) | YES  | UNI | NULL    |                |
+----------------+--------------+------+-----+---------+----------------+
5 rows in set (0.001 sec)
```

Figure B.2: *DESCRIBE output for Member, Membership_Plan, Membership and Trainer*

B.2 Diagram Files

The diagrams in this report are also supplied as separate PNG and SVG files: system architecture, authentication flow, Level-1 data flow, application workflow and entity-relationship diagram.

B.3 Test Evidence Summary

Table B.1: Observed evidence and interpretation

Check	Observed result	Interpretation
Admin login	Successful; dashboard opened.	Local administrator authentication verified.
Dashboard modules	Five Member 2 sections and actions tested manually by the contributor.	Manual testing reported; retain a detailed log if required.
MongoDB ping	`{ ok: 1 }`	Local MongoDB reachable.
MongoDB logs	6 documents; latest attendance event had Attendance ID 14.	Observed count, not forced.
Next.js build	Compile and TypeScript checks passed; 30 static pages generated.	Local build passed.
GitHub Actions	Two successful Build Check runs reported.	CI build passed.
ESLint	Earlier repository-wide run reported remaining issues in other or shared files.	Lint-clean status must not be claimed.
Authentication	Local table and admin configured; teammate uses a separate local database.	Compare schemas and scripts before final integration.

Appendix C: API Route Data-Access Summary

Table C.1 relates each documented route directory to the data stores it is expected to use, based on the module design. It should be verified against the route source files.

Table C.1: Route to data-store summary

Route directory	MySQL objects	MongoDB
`app/api/attendance`	`Attendance` (and `Member` for the member list)	Writes a Check-in document to `activity_logs`
`app/api/assignments`	`Trainer_Assignment`, `Trainer`, `Member`	—
`app/api/payments`	`Payment`, `Membership`, `Member`	—
`app/api/activity-logs`	—	`activity_logs`

Appendix D: Pre-Submission Checklist

Table D.1: Items to complete or confirm before submission

Item	Action
Member 1 responsibilities	Add the documented responsibilities of Priyanka B M (modules and tasks).
Pull Request #1	Confirm review and merge status.
Shared authentication	Compare `auth_users_setup.sql` with the teammate's version.
Payment status column	Confirm whether the column is `status` or `payment_status`.
API route details	Add HTTP methods, request fields and responses from the route files.
Reporting views	Confirm `daily_attendance_report` and `payment_report` with `SHOW FULL TABLES`.
Technology versions	Add React, Node.js, MongoDB and library versions from `package.json`, and confirm whether Tailwind CSS is used.
Open findings	Check the sample email value and the date difference between pages.
Course name	This report uses the course label from the original report (Database Management Systems); change it if required.